

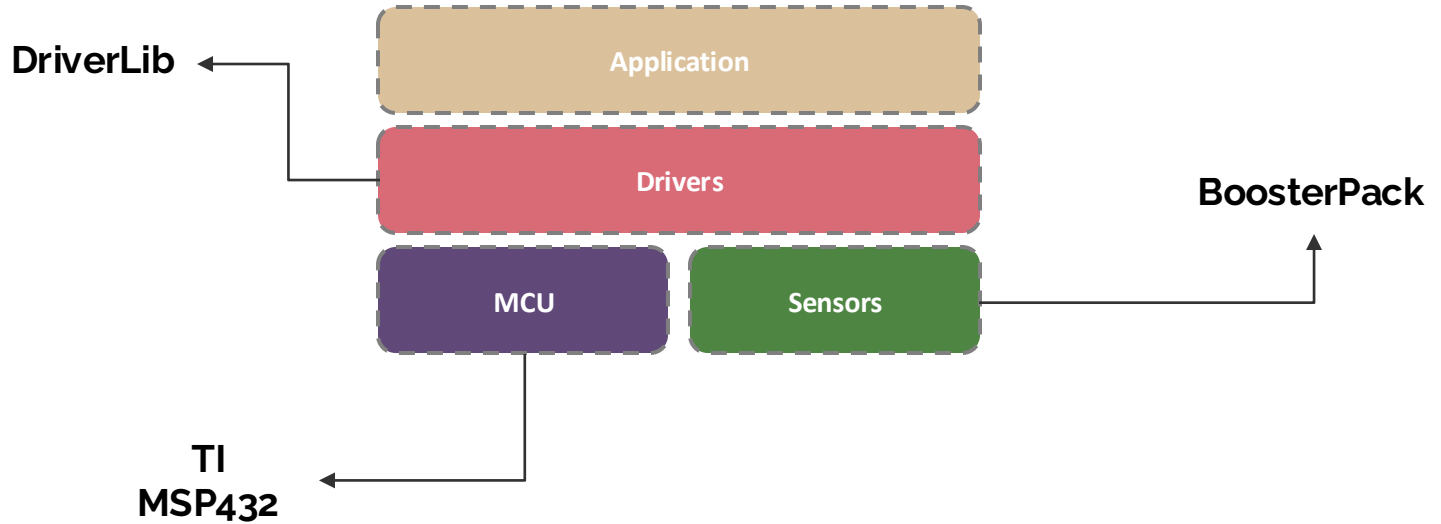


Embedded Software for the Internet Of Things

Introduction to Real Time Concepts

Assoc. Prof. Kasim Sinan Yildirim
University of Trento, Italy

Bare-Metal Software System



Operating Systems

- An intermediary interface between applications and computer hardware
 - Facilitates application development (convenience and efficiency)



Embedded OS

- Designed to be used in embedded computer systems
 - Manages limited resources such as memory, IOs, clocks...
- Should be very **compact** and **energy-efficient**.

Embedded RTOS

- Must quickly and predictably respond to events
 - Responds to requests within a **guaranteed** and **predictable delay**
- Processes data in deterministic cycles

Reactive Systems

- Most embedded systems are **reactive**
 - Respond to external events
- Being reactive means a **real-time response**
 - Response within a predefined time
- How to design a real-time software system architecture?
 - How to implement programs?

Reactive Systems

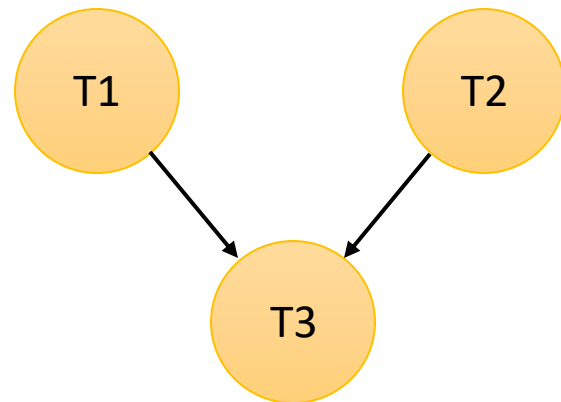
- General-purpose operating systems allocate resources without regard to time
 - They **can miss deadlines**
- Real-time operating systems (RTOSs) allocate resources using algorithms that take real-time into account
 - The target **meeting the deadlines!**

Processes and Operating Systems

- Many applications are sophisticated:
 - A single program can easily become **too complex**
 - Timing constraints make programs **even more complex!**
- Process abstraction and the RTOS help building real-time applications:
 - Switching between processes makes it much easier to **satisfy timing requirements**

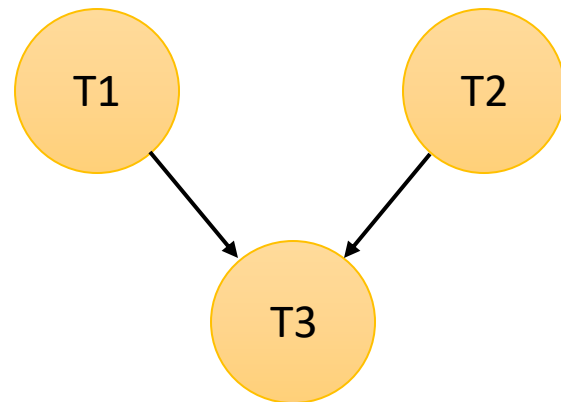
Task (or Process)

- A **real-time application** is a functional description of a set of tasks.
 - We decompose the application into **smaller pieces**, called **tasks**:
 - *Work together in an organized way and may communicate.*
- In the RTOS domain, we use the term **task** instead of the term **process** (i.e., **Task = Process**)



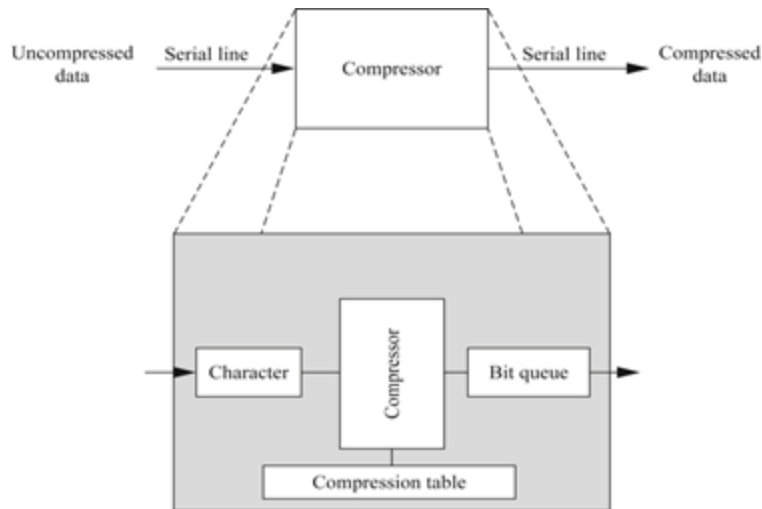
Task (or Process)

- A task (or process) has its **own state**: registers, memory, ..
- The RTOS **manages tasks**:
 - Tasks run in the **same address space** (can also be seen as **threads**).



Why Tasks?

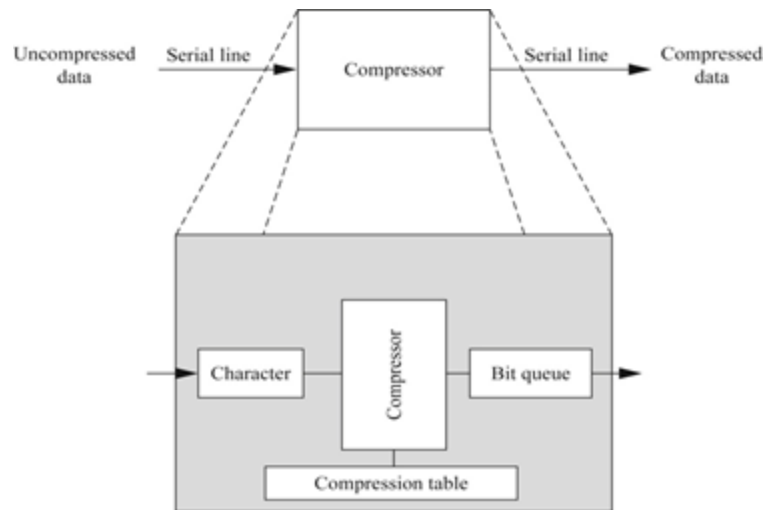
- The compression engine as a **multi-rate system**:
 - receives and sends data at different rates
 - **asynchronous input**: *we do not know when the input will arrive (e.g., user interfaces)*
- When there are several different timing requirements, the control structures **become complex**.



Why Tasks?

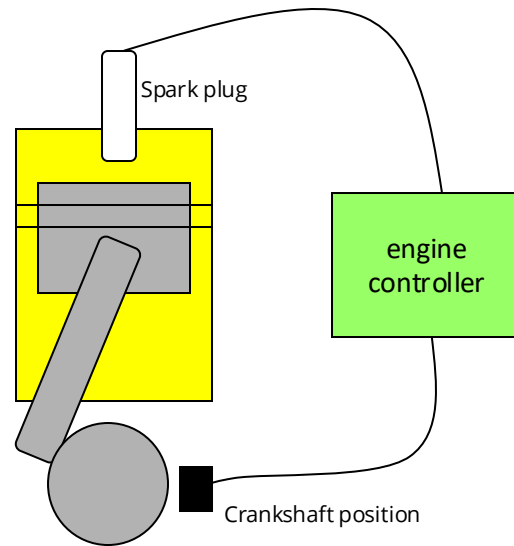
- **Multiple** rates of computation must be handled:

- Handle characters
- Compress data
- Send compressed data



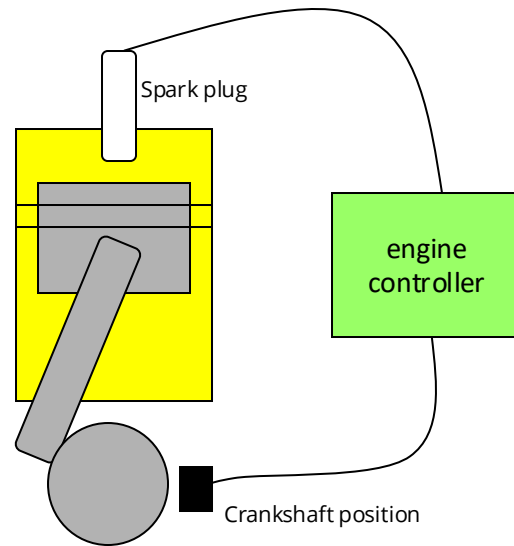
Why Tasks?

- **Engine Control** has several tasks:
 - spark control: fires air/fuel mixture
 - crankshaft sensing: to monitor the position or rotational speed of the crankshaft.
 - *This information is used by engine management systems to control the fuel injection or other engine parameters.*
 - fuel/air mixture
 - oxygen sensor
 - kalman filter: algorithm for estimation and control



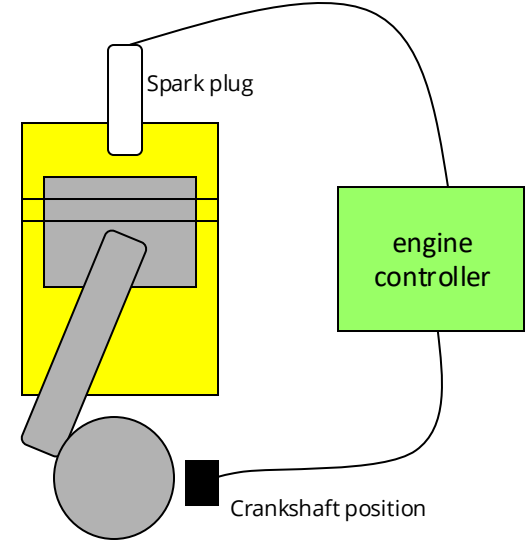
Why Tasks?

- Firing the spark plug is a **periodic process**:
 - fired at a certain point in the combustion cycle
- Using a microcontroller that senses the engine crankshaft position:
 - Allows the spark timing to vary with engine speed.
- Additional sensors:
 - gas pedal position, oxygen sensor used to control emissions



Why Tasks?

Variable	Update period (ms)
Engine spark timing	2
Throttle	2
Air flow	4
Battery voltage	4
Fuel flow	10
Recycled exhaust gas	25
Status switches	20
Air temperature	400
Barometric pressure	1000
Spark (dwell)	1
Fuel adjustment	8
Carburetor	25
Mode actuators	100

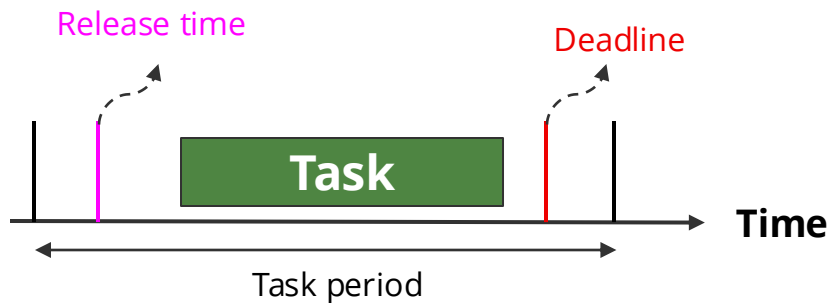
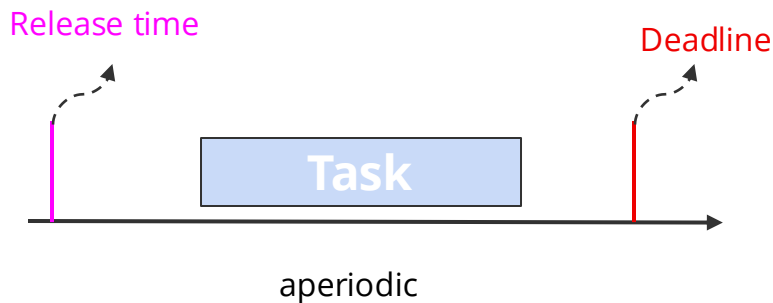


Real-Time Systems

- Perform tasks to conform to **external timing constraints**.
 - Real-time **guarantees** the completion of a process within a defined time interval.
 - Does not make any statement about the **total time duration** or the **processing speed**.

Real-Time Systems

- **Release time:** the time at which the task becomes **ready**.
- **Deadline:** the time at which the task **must be finished**.
 - **Periodic:** The deadline is some time other than the end of the period.
 - **Aperiodic:** Deadline is measured from the release (initiation) time

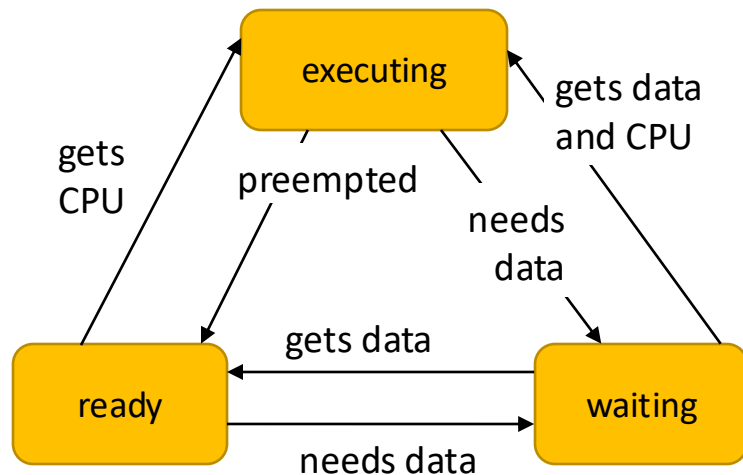


Timing Violations

- **Jitter:** the variability of the time it takes to complete a task
- What happens if a task isn't finished by its deadline?
 - **Hard deadline:** system fails if missed.
 - **Soft deadline:** the user may notice, but the system doesn't necessarily fail.
- Take a variety of actions when a deadline is missed.
 - *Safety-critical systems:* e.g., take compensatory measures such as switching into a special safety mode.
 - *Systems for which safety is not as important:* e.g., completely ignore the failure.

RTOS Scheduling

- RTOS determines *the task that runs next*.
 - Choosing the order of running tasks is known as **scheduling**.
- A task can be in one of three **states**:
 - executing on the CPU, ready to run, waiting for data
- **Scheduling algorithm**:
 - How processes are selected from the ready state to the running state.



RTOS Scheduling

- Various scheduling algorithms exist:
 - **Cooperative scheduling:** no pre-emption, tasks can end themselves in a cooperative manner
 - **Preemptive scheduling:** tasks can be interrupted by other tasks of higher priorities
 - **Earliest deadline first approach:** the task with the earliest deadline is served first

RTOS Scheduling

- Event-driven RTOS
 - **Preemptive task scheduling**
 - *An event of higher priority needs to be served first*
 - *Processes data more responsively*

The Scheduling Problem

- Can we meet all deadlines?
 - How much CPU power do we need to meet our deadlines?
- The scheduling process also consumes CPU time.
 - Not all CPU time is available for processes.
 - Scheduling overhead must be taken into account for the exact schedule.

A Path to the Operating Systems

Let's start with the basics to show why we need an RTOS!

While Loop Implementation

- Here is a very simple program that runs our process subroutines repeatedly:

```
while (TRUE) {  
  p1();  
  p2();  
}
```

- Simplest implementation (has just one loop).
 - **No control over execution timing:**
 - *It does not control the rate at which the processes are executed*
 - How long p1() is executed?

Tasks in ISRs?

- Why not just put the tasks into the timer handler?

```
void interrupt timer_handler() {  
    if (TOIE && TOIF) { /* timer 0 interrupt */  
        task1();  
        task2();  
        task3();  
        TOIF = 0; /* clear timer 0 interrupt flag */  
    }  
}
```

Tasks in ISRs?

- Why not just put the tasks into the timer handler?
- If the tasks we executed in timer_handler() **takes too long**:
 - The timer stops the execution of the previous iteration.
 - The interrupted task may be left in an **inconsistent state**.

```
void interrupt timer_handler() {  
    if (TOIE && TOIF) { /* timer 0 interrupt */  
        task1();  
        task2();  
        task3();  
        TOIF = 0; /* clear timer 0 interrupt flag */  
    }  
}
```

Introduction to Embedded RTOS

OS Structure

- OS needs to keep track of:
 - task priorities
 - scheduling state
 - **task activation record**
- Tasks may be created:
 - statically before system starts
 - dynamically during execution

Embedded vs General Purpose Scheduling

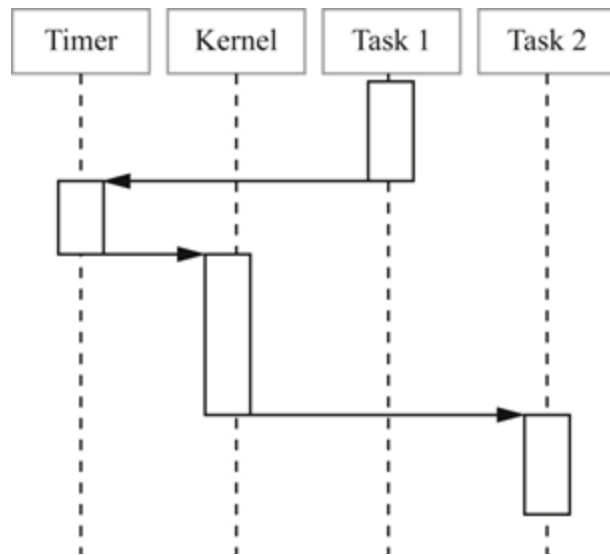
- Workstations try to avoid starving processes of CPU access.
 - Fairness = access to CPU.
- Embedded systems must meet deadlines.
 - **Low-priority processes should not run for a long time!**

Preemptive RTOS

- The most reliable way to meet timing requirements accurately is to build a **preemptive operating system**:
 - It executes tasks based on timing requirements *provided by the system designer*.
 - Timing requirements are expressed as **task priorities** to control what process runs at any given time.

Preemptive Scheduling

- **Timer interrupt** gives CPU to **kernel**.
 - **Time quantum:** the length of the timer period
 - *the smallest increment in which we can control CPU activity*
- Kernel is a part of the operating system:
 - determines what process is running
 - decides what task runs next
 - performs a **context switch** to the new context



Context Switching

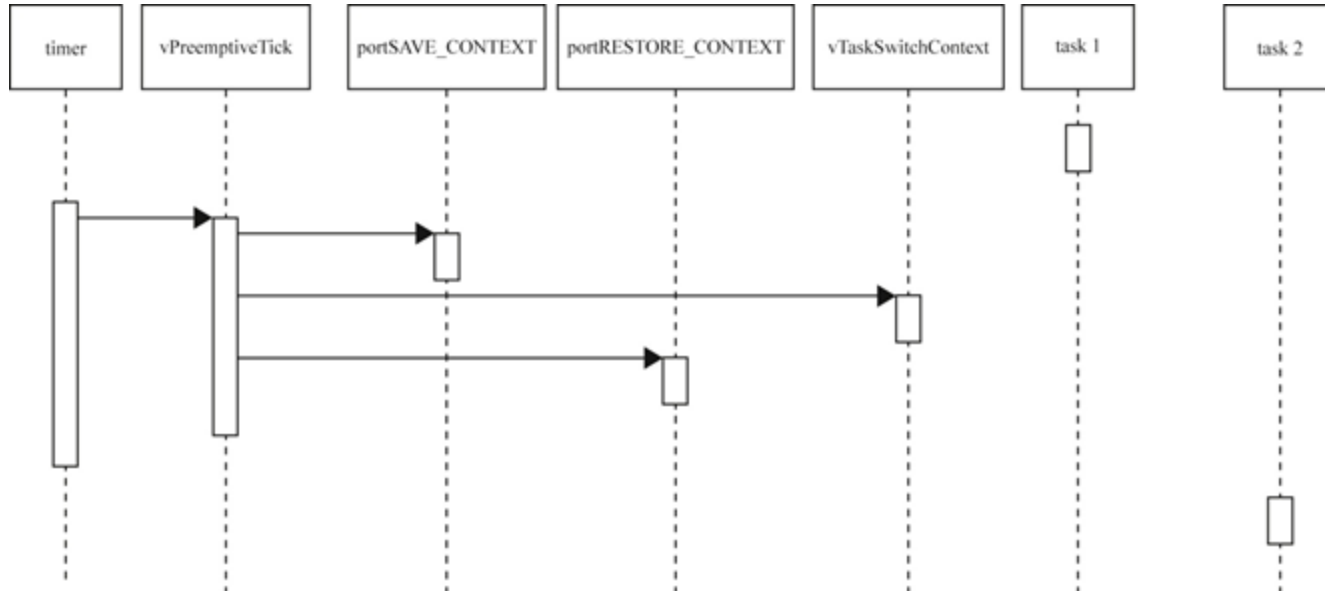
- How do we switch between tasks before the task is done?
 - We cannot rely on C-level mechanisms to do so.
 - **Use assembly language** to switch between processes.
- **Context switching:**
 - Switching from one task's register set to another
 - **Context:** *the set of registers that defines a task*
- **Record:**
 - the data structure that holds the state of the task

FreeRTOS

- The best way to understand processes and context is to dive into an RTOS implementation.
 - FreeRTOS.org kernel as an example
 - *version 7.0.1 for the ARM7 AVR32 platform.*
- A process is known in FreeRTOS as a task.

FreeRTOS Context Switch

- Let us start with the steady state:
 - everything has been initialized, the operating system is running, and we are ready for a timer interrupt.



vPreemptiveTick() IN portISR.c

- Calls a set of system routines:
 - **vPreemptiveTick()** - called when the timer ticks.
 - **portSAVE_CONTEXT()** - Swaps out the current task context.
 - **vTaskIncrementTick()** - updates the time
 - **vTaskSwitchContext()** - chooses a new task.
 - **portRESTORE_CONTEXT()** - swaps in the new context.

```
void vPreemptiveTick( void )
{
    /* Save the context of the current task. */
    portSAVE_CONTEXT();
    /* Increment the tick count - this may wake a task. */
    vTaskIncrementTick();
    /* Find the highest priority task ready to run. */
    vTaskSwitchContext();
    /* End the interrupt in the AIC. */
    AT91C_BASE_AIC->AIC_EOICR = AT91C_BASE_PITC->PITC_PIVR;
    portRESTORE_CONTEXT();
}
```

portSave_Context() in PORTMACRO.h

- The context includes the 32 general-purpose registers, PC, status register, and stack pointers SPH and SPL.
- Next, the kernel stores the stack pointer.

```
push r0
in r0, __SREG__
cli
push r0
push r1
clr r1
push r2
...
; continue pushing all the registers
...
push r31
lds r26, pxCurentTCB
lds r27, pxCurentTCB + 1
in r0, __SP_L__
st x+, r0
in r0, __SP_H__
st x+, r0
```

vTaskSwitchContext() in tasks.c

- Walks down the list of tasks to identify the **highest-priority task**.

```
void vTaskSwitchContext( void ){
    if( uxSchedulerSuspended != (unsigned portBASE_TYPE) pdFALSE){
        /* The scheduler is currently suspended - do not allow a context switch. */
        xMissedYield = pdTRUE;
    }
    else {
        traceTASK_SWITCHED_OUT();
        taskFIRST_CHECK_FOR_STACK_OVERFLOW();
        taskSECOND_CHECK_FOR_STACK_OVERFLOW();
        /* Find the highest priority queue that contains ready tasks. */
        while( listLIST_IS_EMPTY(&(pxReadyTasksLists[uxTopReadyPriority]))){
            configASSERT(uxTopReadyPriority);
            --uxTopReadyPriority;
        }
        /* listGET_OWNER_OF_NEXT_ENTRY walks through the list, so the tasks of the
        same priority get an equal share of the processor time. */

        listGET_OWNER_OF_NEXT_ENTRY(pxCurrentTCB,&(pxReadyTasksLists[uxTopReadyPriority]));
        traceTASK_SWITCHED_IN();
        vWriteTraceToBuffer();
    }
}
```

portRestore_Context() in PortMacro.h

- First loads the address for the new task's stack pointer
- Then, gets the stack pointer register values
- Finally, restores the general-purpose and status registers.

```
lds r26, pxCurrentTCB
lds r27, pxCurrentTCB + 1
ld r28, x+
out __SP_L__, r28
ld r29, x+
out __SP_H__, r29
pop r31
; pop the registers
pop r1
pop r0
out __SREG__, r0
pop r0
```

Priority-driven Scheduling

- Each task has a priority.
 - CPU goes to the **highest-priority task** that is ready.
- Priorities determine scheduling policy:
 - fixed priority
 - time-varying priorities.

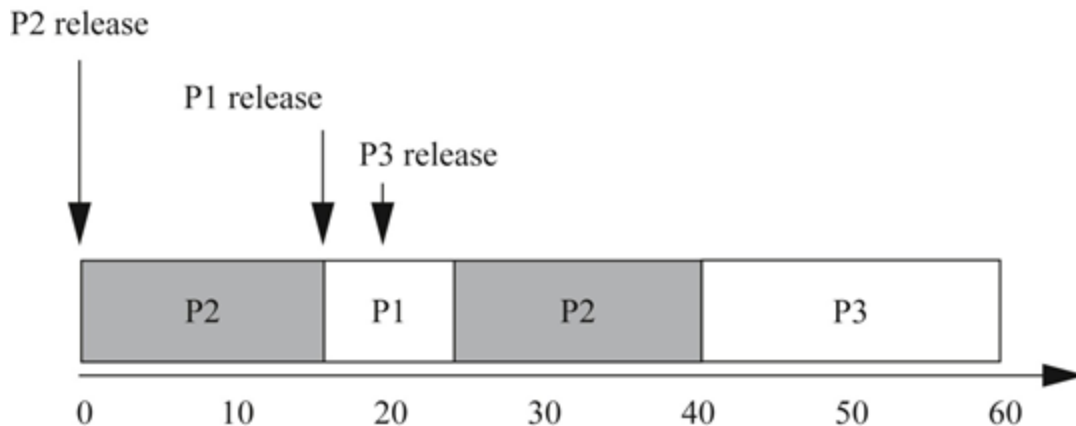
Priority-driven Scheduling - Example

- Rules:
 - Each task has a fixed priority (1 highest);
 - Highest-priority ready task gets CPU
 - The task continues until done.

Priority-driven Scheduling - Example

- Tasks

- P1: priority 1, execution time 10
- P2: priority 2, execution time 30
- P3: priority 3, execution time 20



RTOS Scheduling Algorithms

Rate Monotonic Scheduling

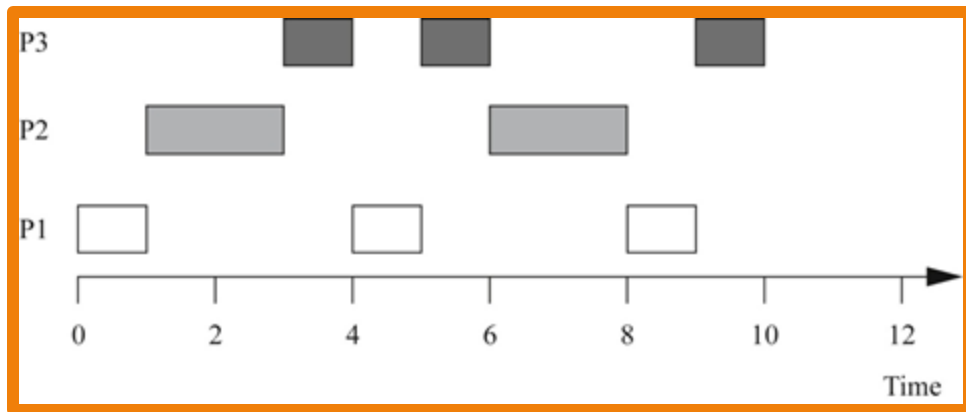
- A static scheduling policy:
 - it assigns fixed priorities to processes
 - fixed priorities are sufficient to efficiently schedule the processes

Rate Monotonic Scheduling

- Assumptions:
 - Zero context switch time, no data dependencies between processes, process execution time is constant, and the deadline is at the end of the period.
- The highest-priority ready process is always selected for execution.
 - Priorities are assigned by **rank order of period**
 - *The process with the shortest period is assigned the highest priority.*

RMS Example

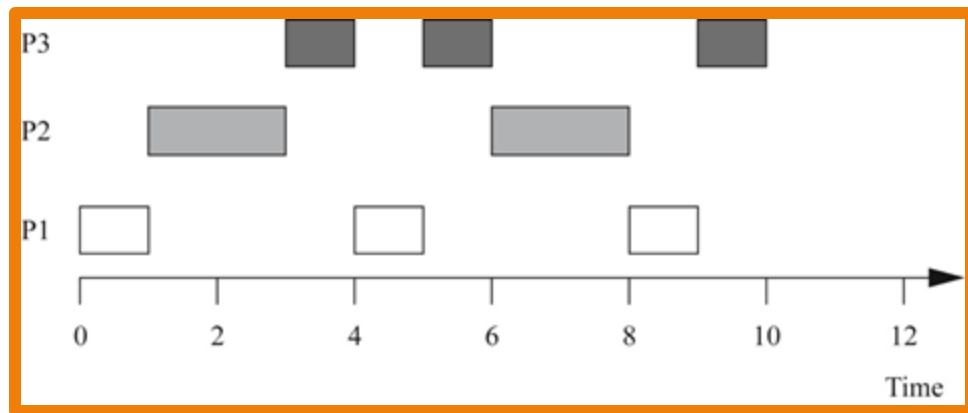
- All three tasks start at time zero.
 - P1 - Execution Time:1 Period: 4
 - P2 - Execution Time:2 Period: 6
 - P3 - Execution Time:3 Period: 12
- Construct a timeline equal in length to **the least common multiple** of the process periods
 - **Hyperperiod:** 12 in this case.



RMS Example

- There is **no feasible assignment of priorities** that guarantees scheduling:

- We must execute P1 three times, requiring 6 units of CPU time
- P2 twice, costing 6 units of CPU time,
- P3 one time, requiring 3 units of CPU time.
- The total of $6 + 6 + 3 = 15 > 12$ time units.



Exceeding the available CPU capacity

CPU Utilization

- RMS **does not allow** the use of 100% of the available CPU cycles.
- Example:
 - P1 has a period of 4 and an execution time of 2
 - P2 has a period of 7 and an execution time of 1
 - The **hyper period** of the processes is **$4 \times 7 = 28$**
 - P1 runs 7 times and P2 runs 4 times.
 - The **CPU utilization** of this set of processes: $[(2 \times 7) + (1 \times 4)]/28 = 0.64$

RMS Implementation

- Efficient implementation:
 - Sort array at the beginning
 - Scan processes
 - Choose the highest-priority active process.

```
/* processes[] is an array of process activation records, stored
in order of priority, with processes[0] being the highest-
priority process */
Activation_record processes[NPROCESSES];

void RMA(int current) { /* current = currently executing
process */
    int i;
    /* turn off current process (may be turned back on) */
    processes[current].state = READY_STATE;

    /* find process to start executing */
    for (i = 0; i < NPROCESSES; i++)
        if (processes[i].state == READY_STATE) {
            /* make this the running process */
            processes[i].state == EXECUTING_STATE;
            break;
        }
}
```

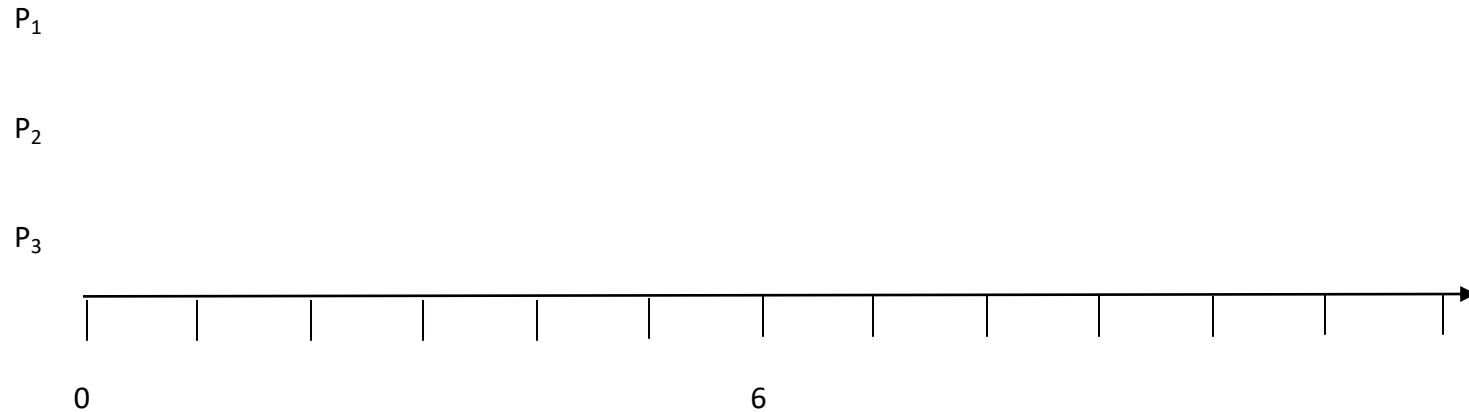
Earliest-Deadline-First Scheduling

- EDF: **dynamic** priority scheduling scheme.
 - The process closest to its deadline has the highest priority.
- Requires the **recalculation of priorities** at every timer interrupt.
- EDF can use 100% of the CPU.
 - But may **fail to miss a deadline**.

EDF Implementation

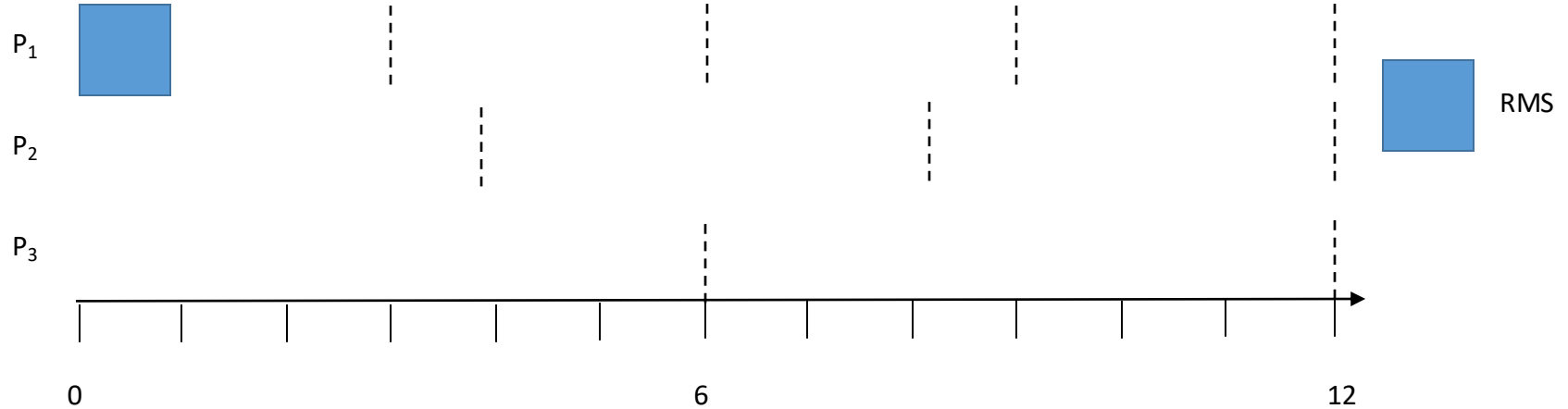
- On each timer interrupt:
 - compute time to deadline
 - choose the process closest to the deadline
- Generally considered **too expensive** to use in practice.

RMS vs EDF



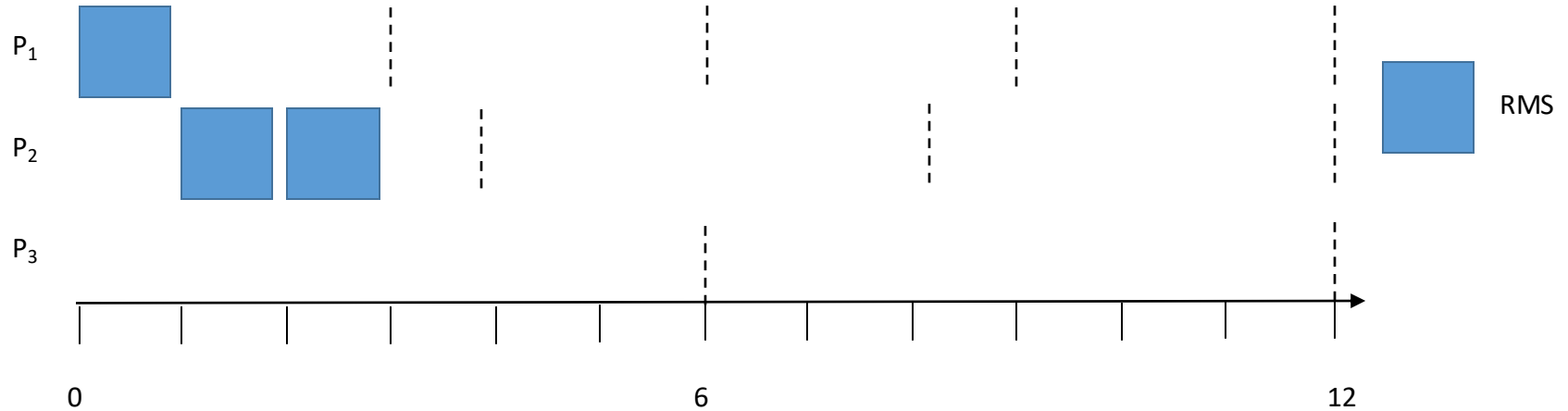
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



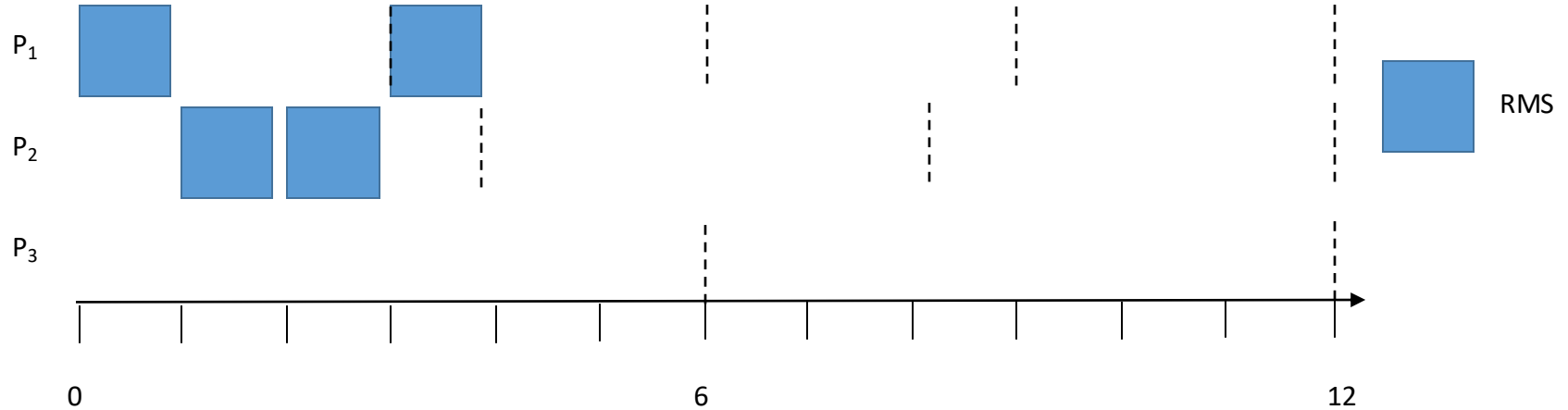
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



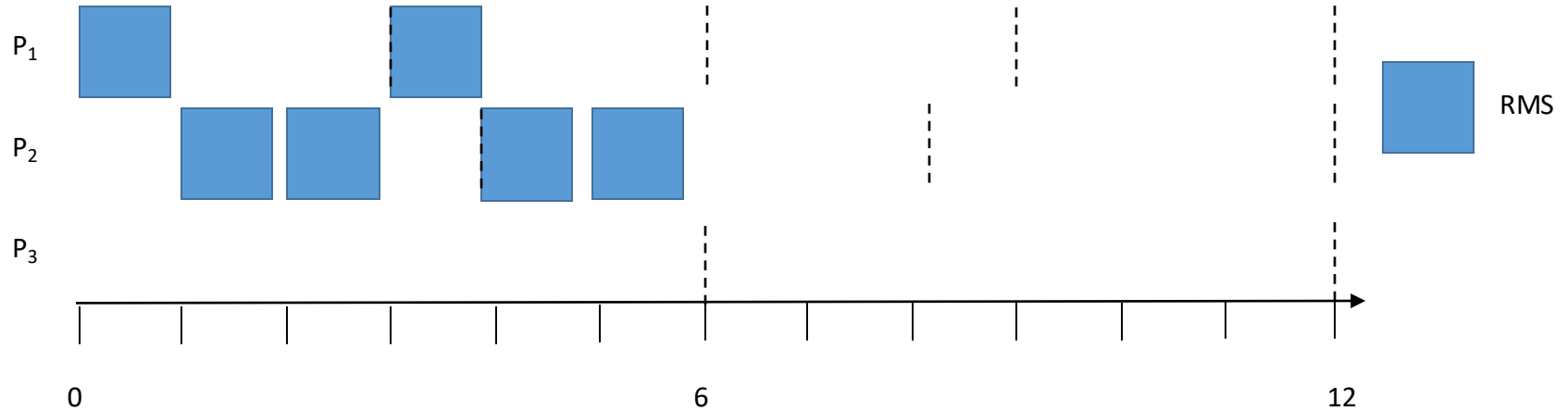
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



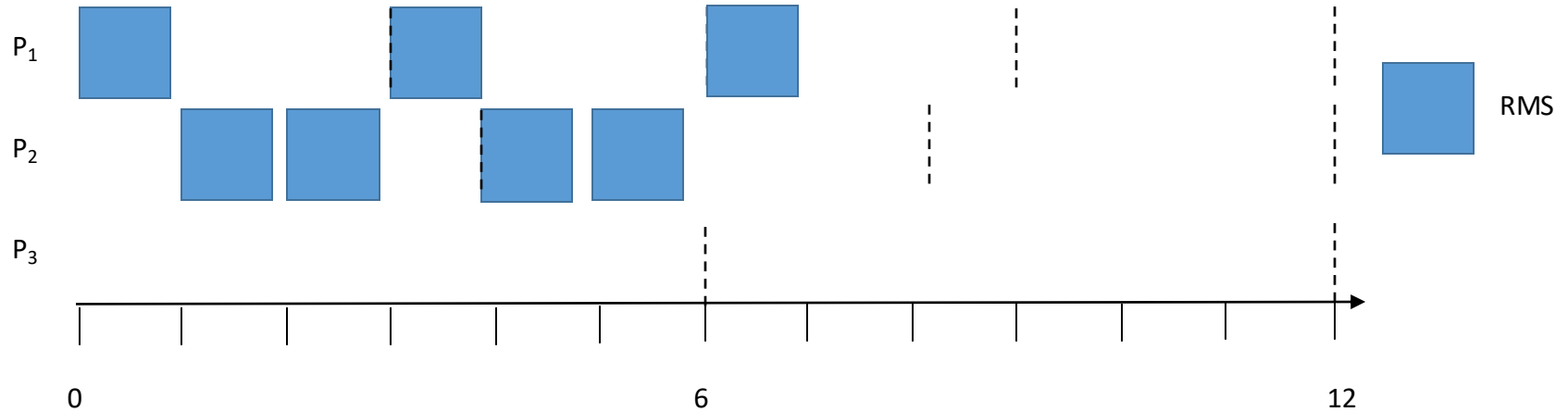
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

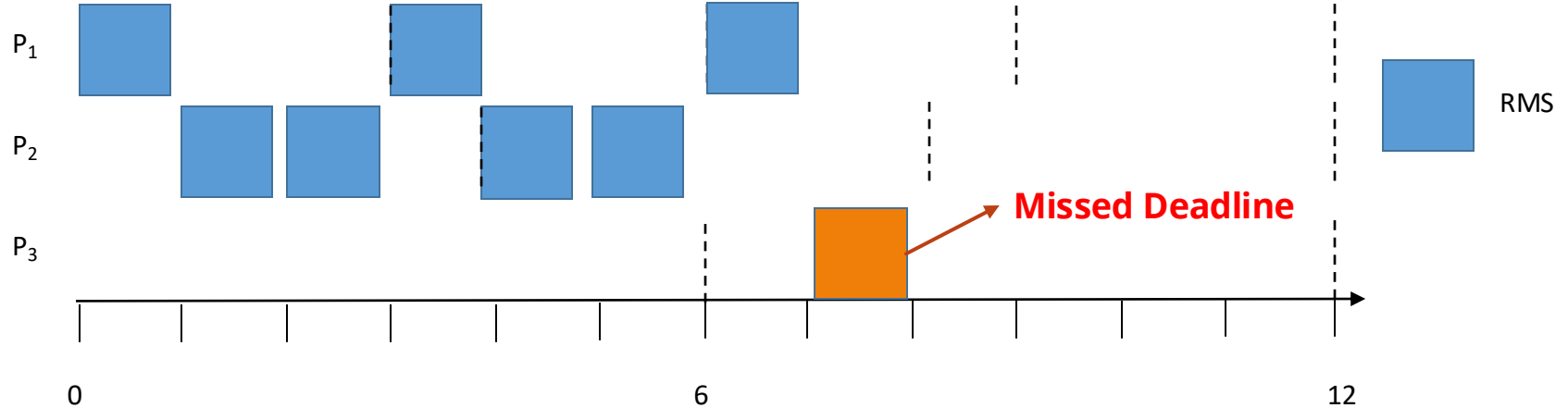
RMS vs EDF



P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

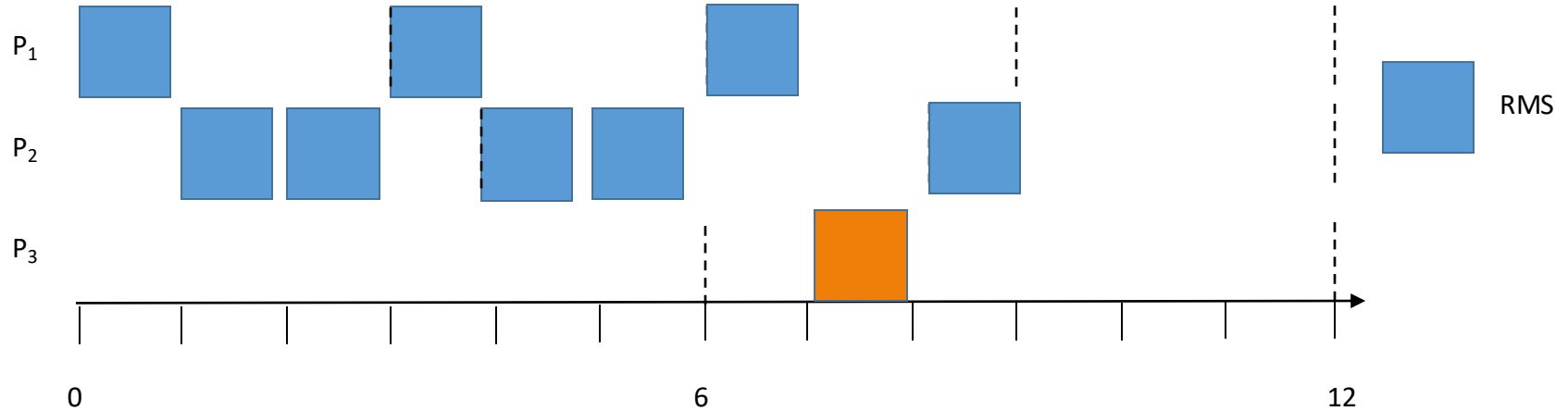
RMS vs EDF

RMS misses P3's deadline at time



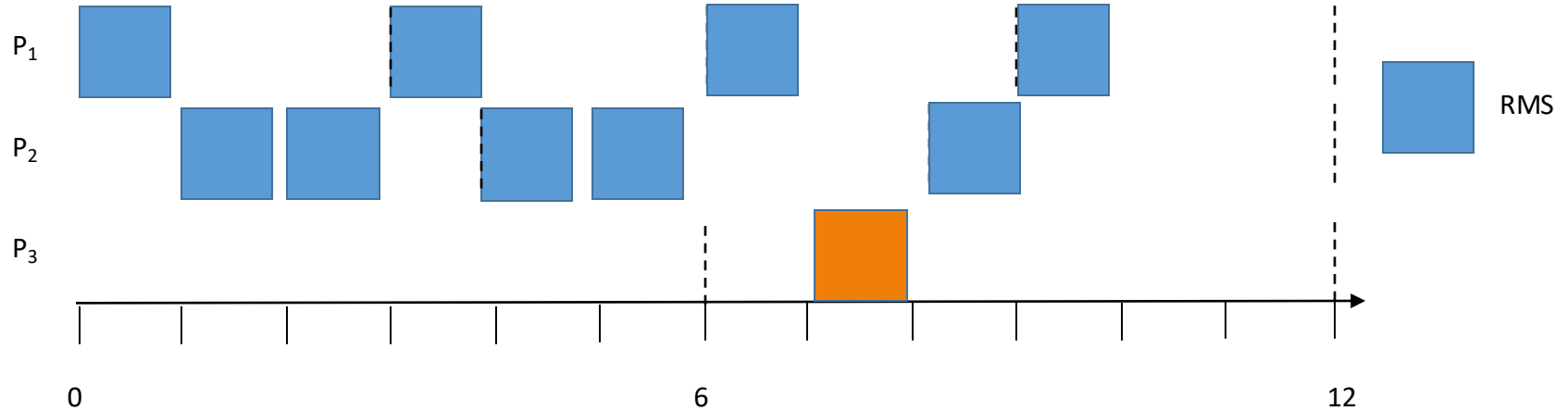
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



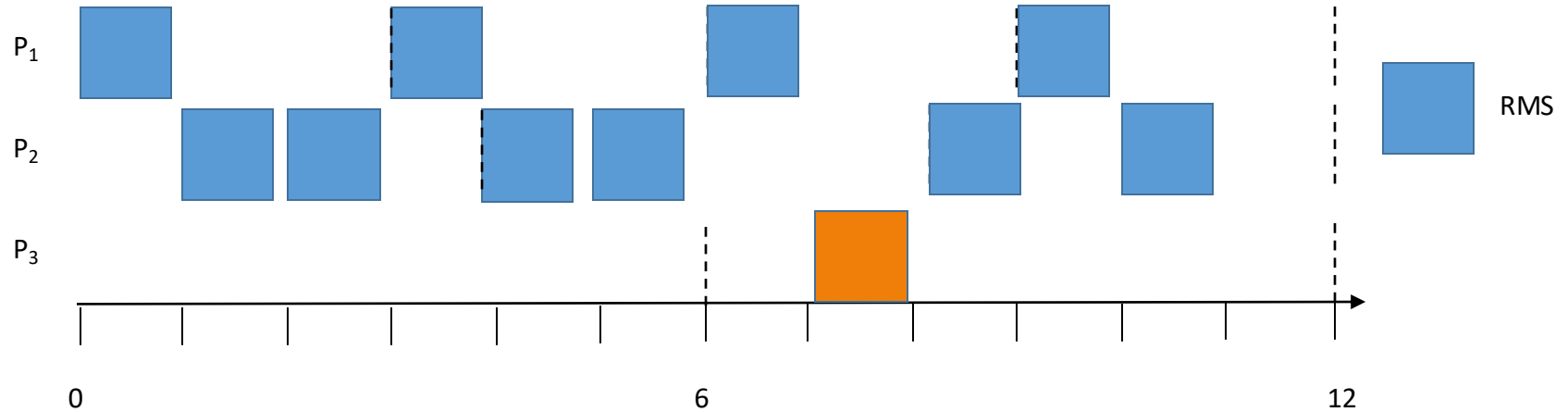
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



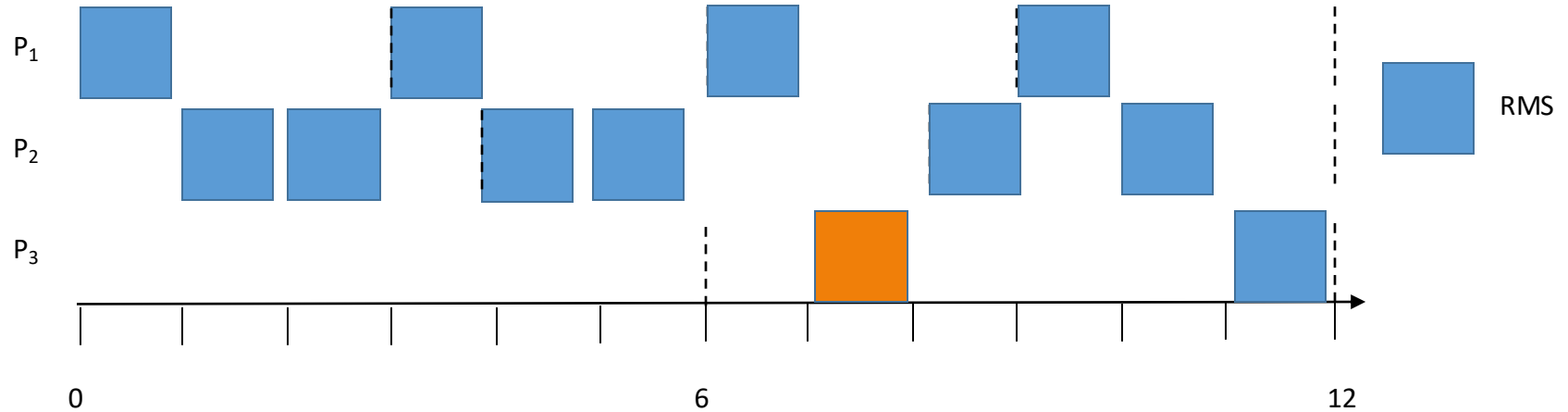
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



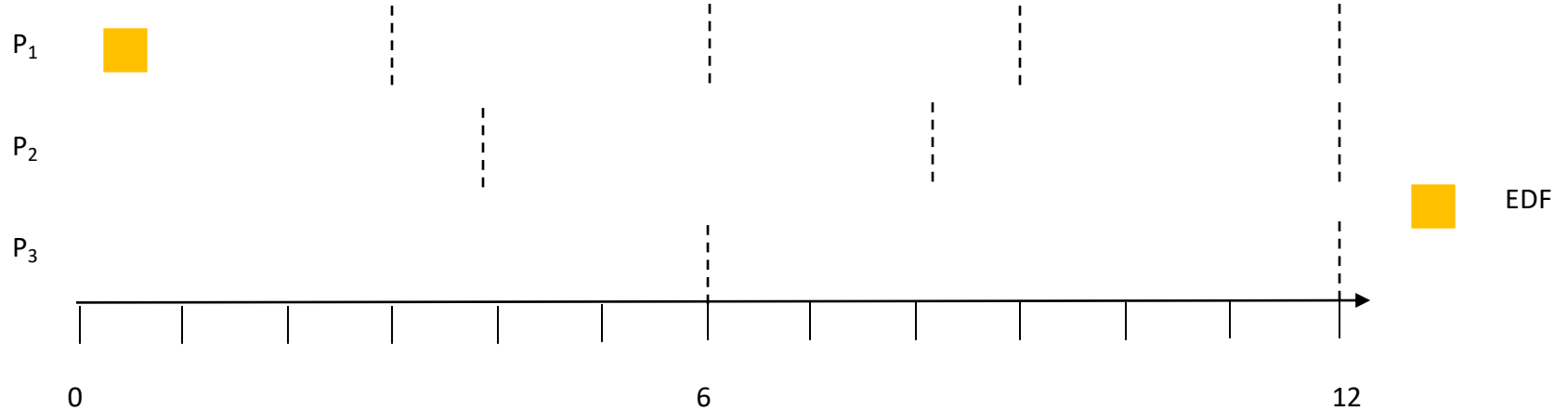
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



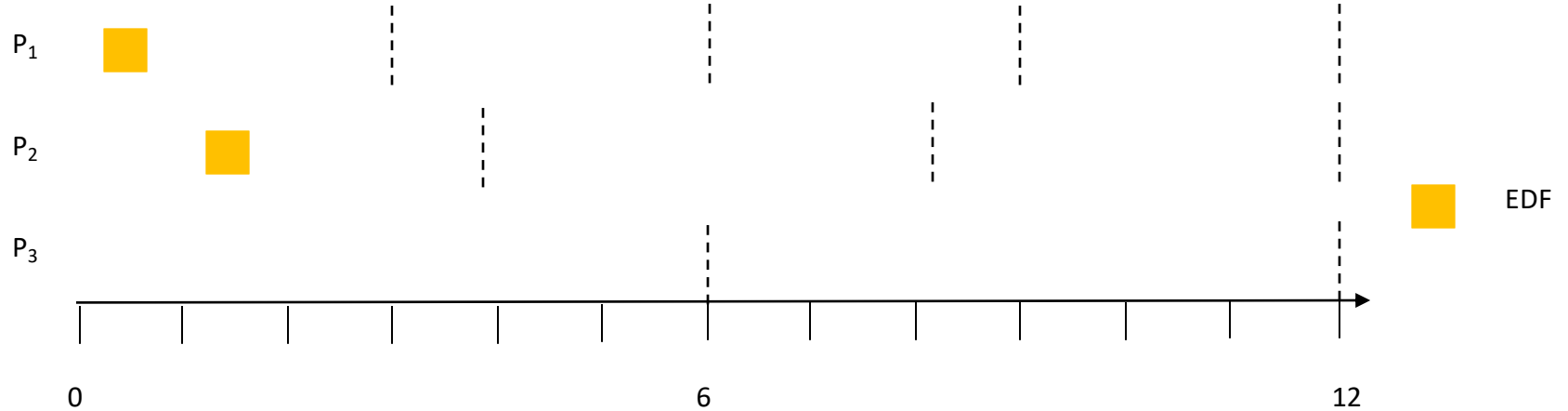
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



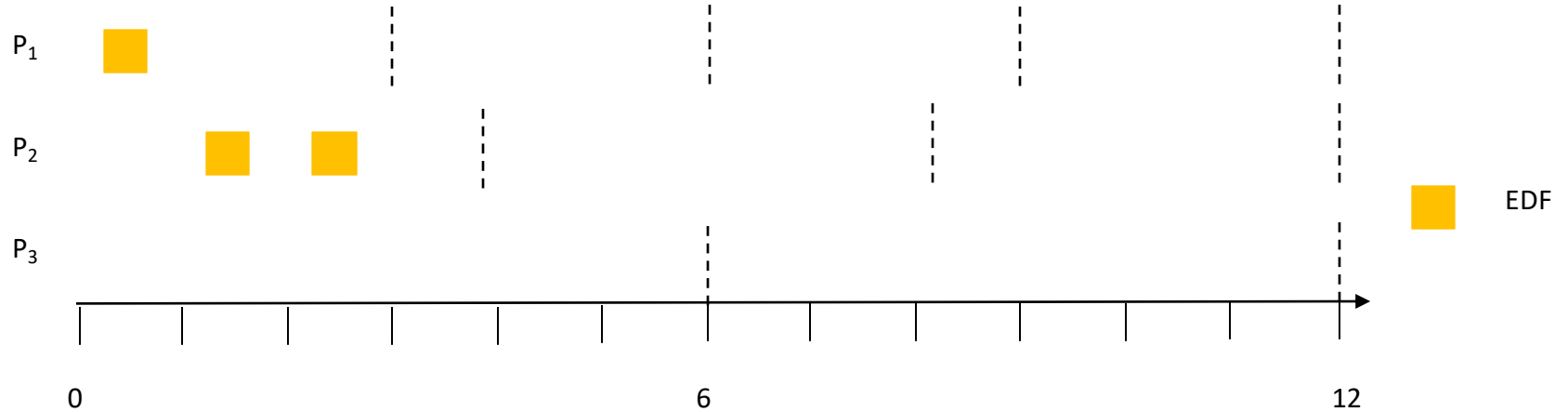
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



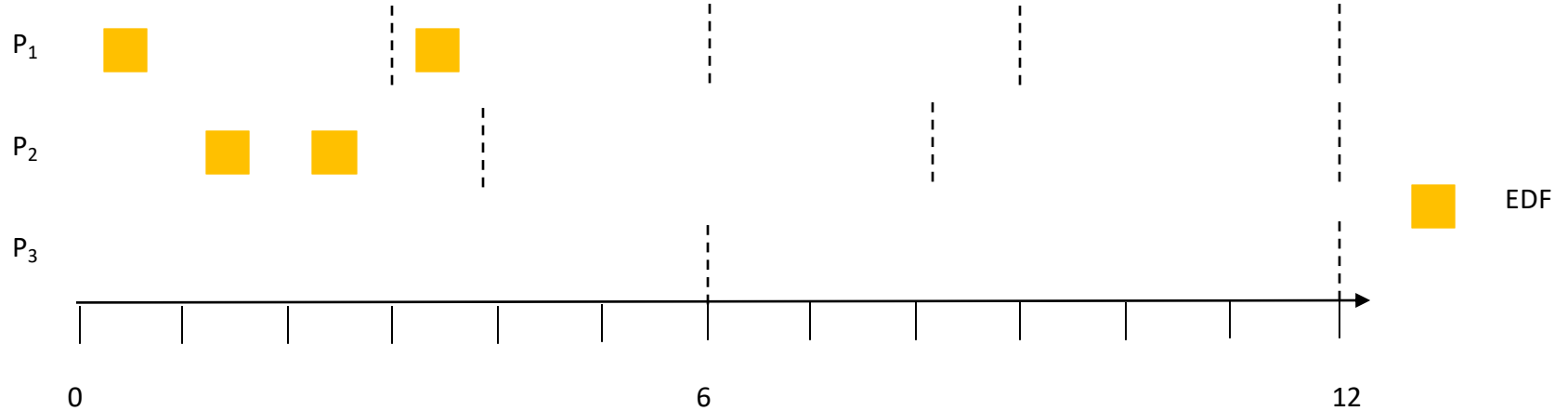
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



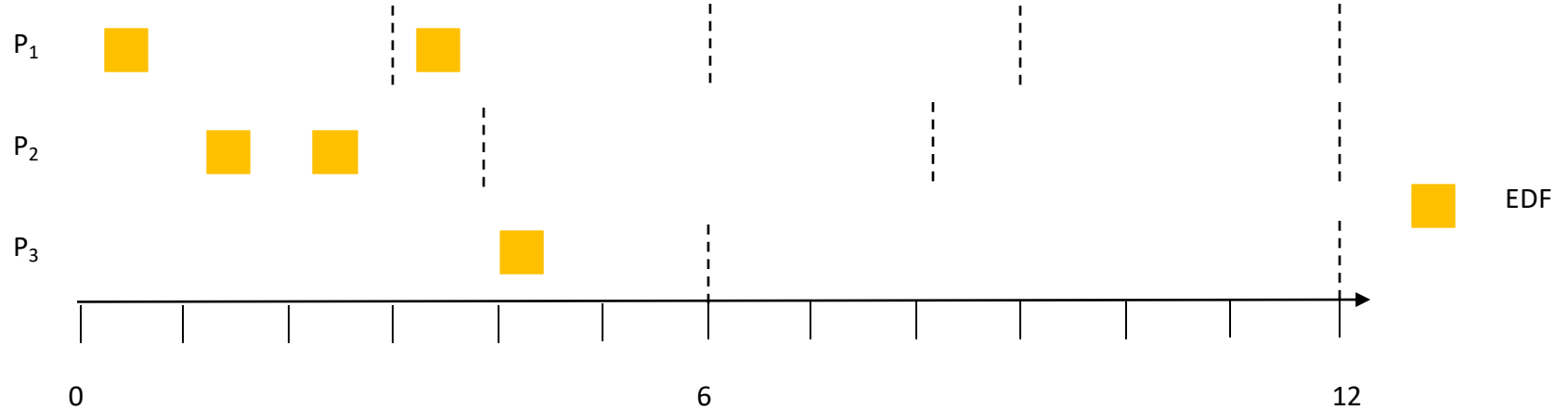
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



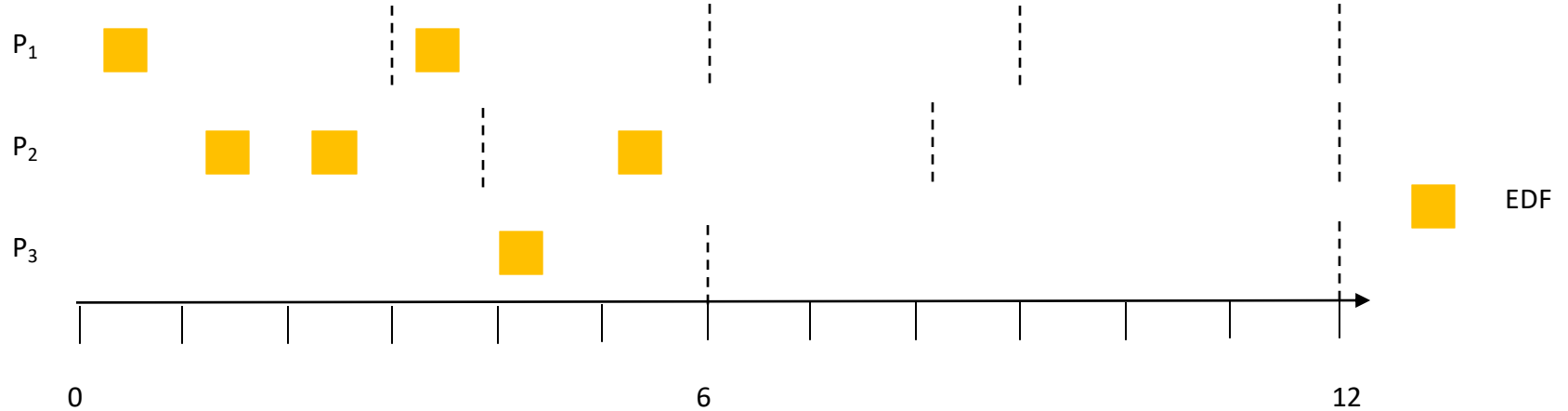
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



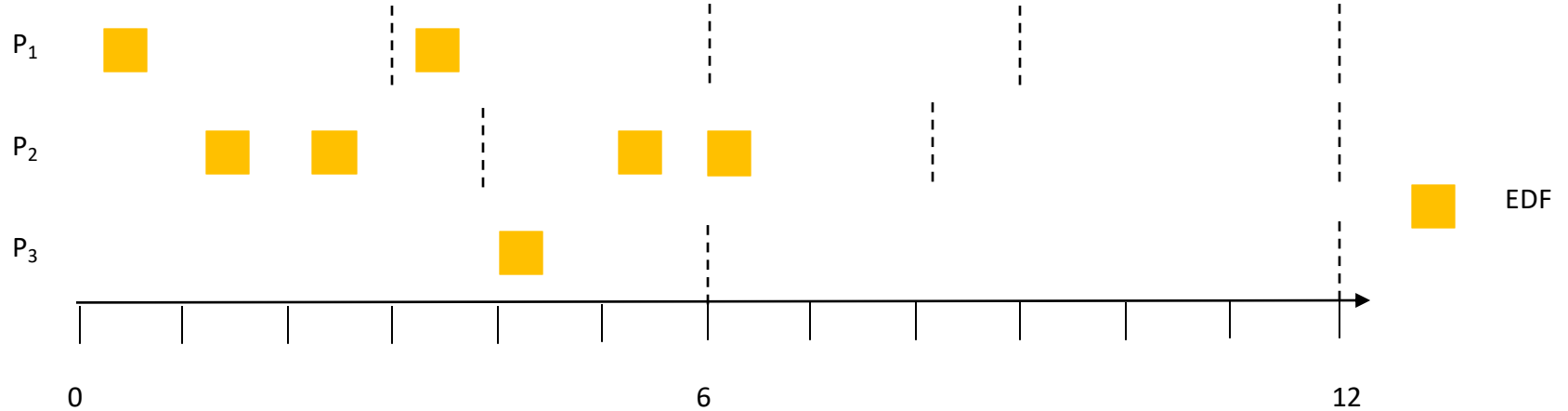
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



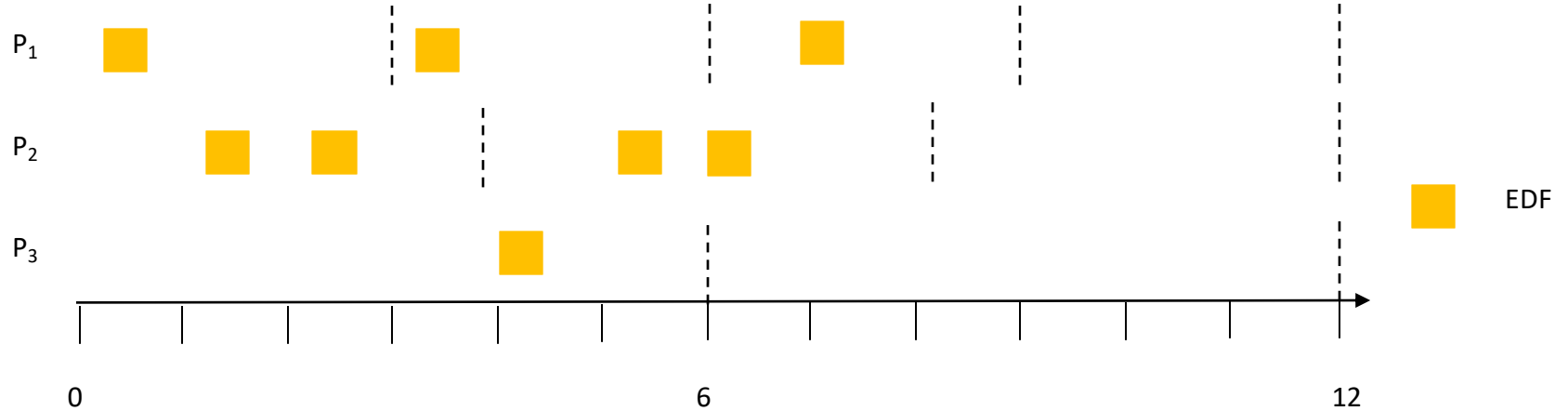
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



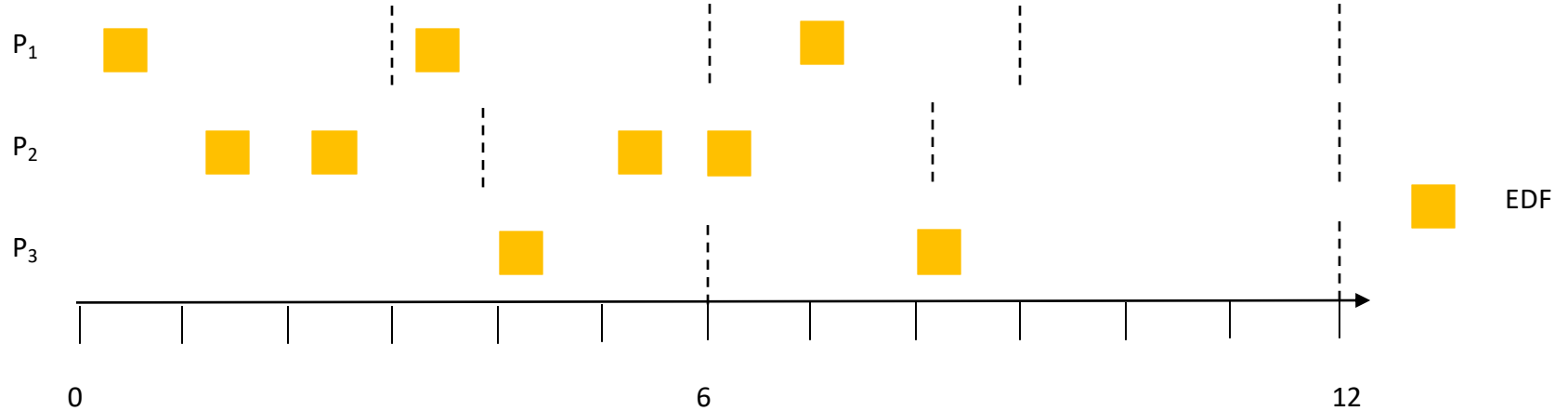
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



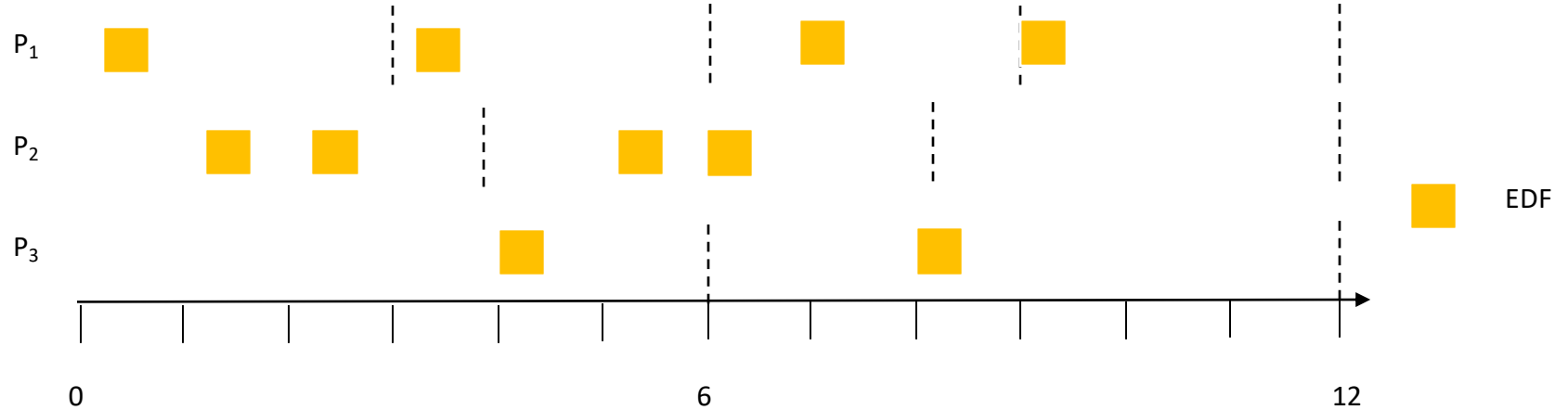
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



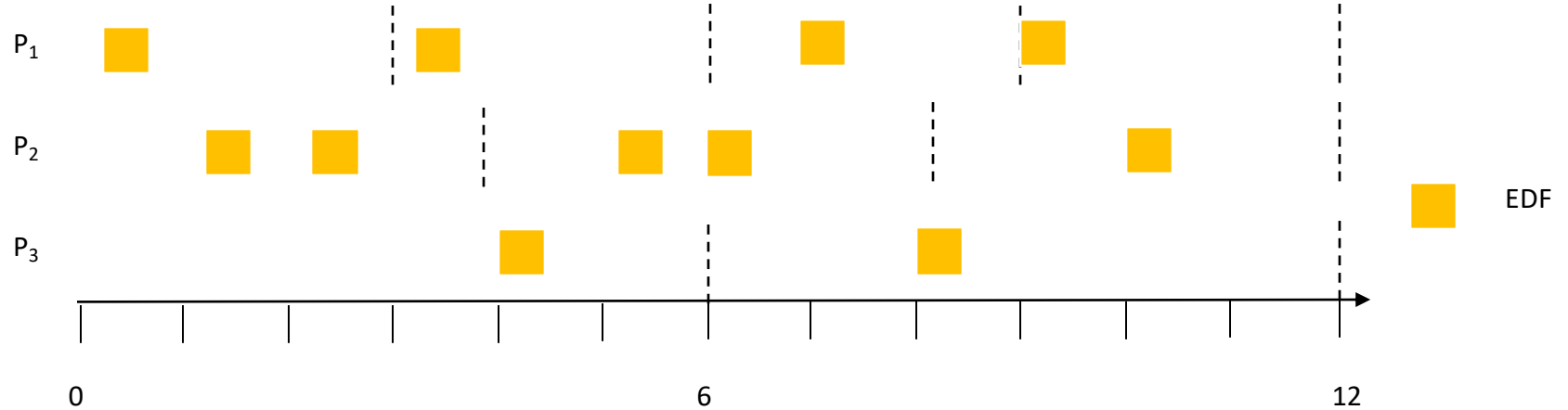
P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF



P₁ - Execution Time:1 Period: 3
P₂ - Execution Time:2 Period: 4
P₃ - Execution Time:1 Period: 6

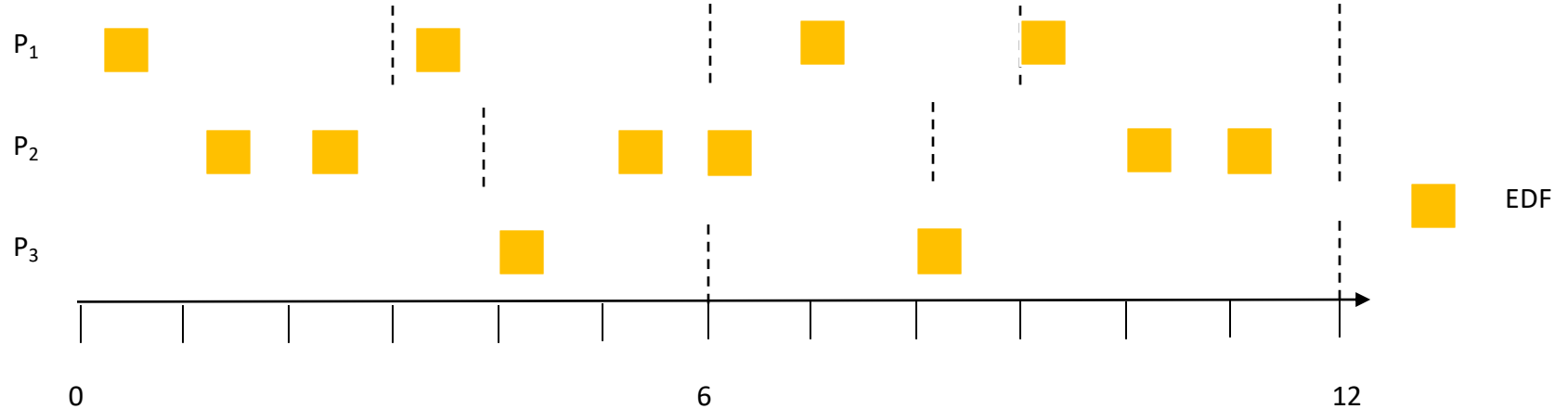
RMS vs EDF



P1 - Execution Time:1 Period: 3
P2 - Execution Time:2 Period: 4
P3 - Execution Time:1 Period: 6

RMS vs EDF

EDF successfully schedules the tasks with a utilization of 100%.



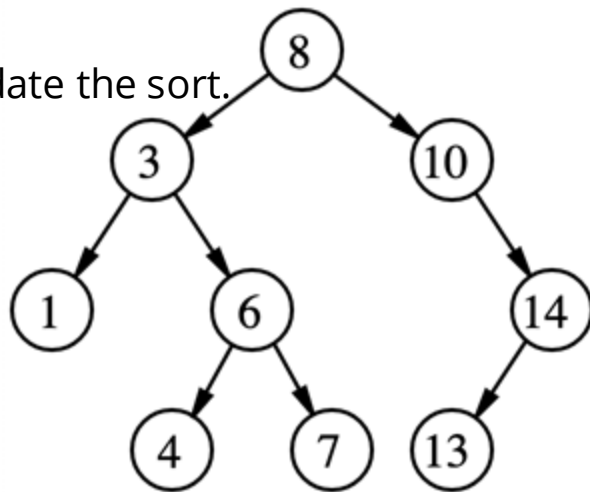
P₁ - Execution Time:1 Period: 3
P₂ - Execution Time:2 Period: 4
P₃ - Execution Time:1 Period: 6

EDF Implementation

- The implementation of EDF is more complex than the RMS code.
- We **cannot presort** the processes into an array, as we could for RMS.
 - Because the times to deadlines for the processes **change** during execution.

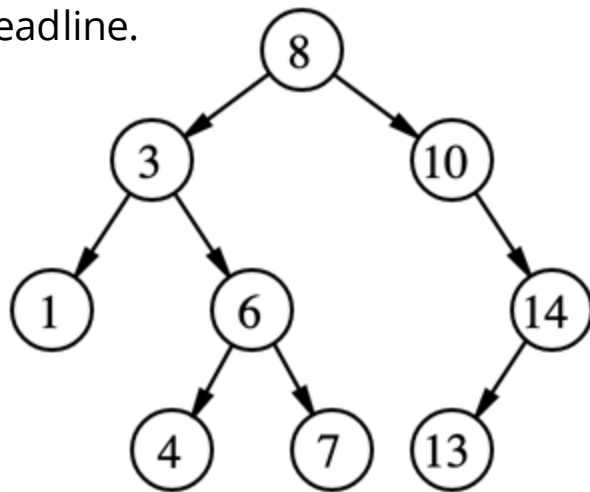
EDF Implementation

- To **avoid re-sorting** the entire set of records at every change, we can build a **binary tree**:
 - keep the deadlines **always sorted** and *incrementally* update the sort.



EDF Implementation

- We need to **update deadlines** at each timer tick:
 - **Traverse** the whole tree and **decrease** the time to the deadline.
 - **Delete** the highest priority and **add** it to the tree again.



Pseudocode for EDF

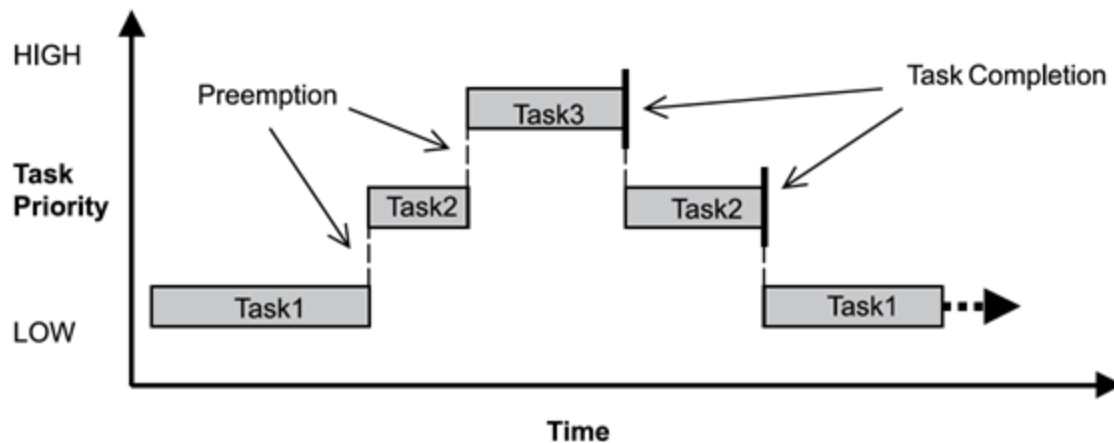
- The dynamic sorting adds **complexity** to the entire scheduling process.
 - Each update of the sorted list requires **$O(n \log n)$** steps.

```
/* process activation records (binary tree) */
Activation_record *processes;

void EDF(Activation_record * current){/*currently executing process*/
    /* turn off current process (may be turned back on) */
    current->state = READY_STATE;
    /* at each tick, update time to deadline */
    updateAndSortDeadlines(processes);
    current = getHighestPriority(deadlines);
    if (current->state == READY_STATE && current != NULL) {
        /* make this the running process */
        current->state == EXECUTING_STATE;
    }
}
```

Preemption Revisited

- Note that Both RMS and EDF are preemptive.



Fixing Scheduling Problems

- What if your set of processes is **not schedulable**?
 - Change deadlines in requirements.
 - Reduce execution times of processes.
 - Get a faster CPU.

Some Practical Aspects

System Performance

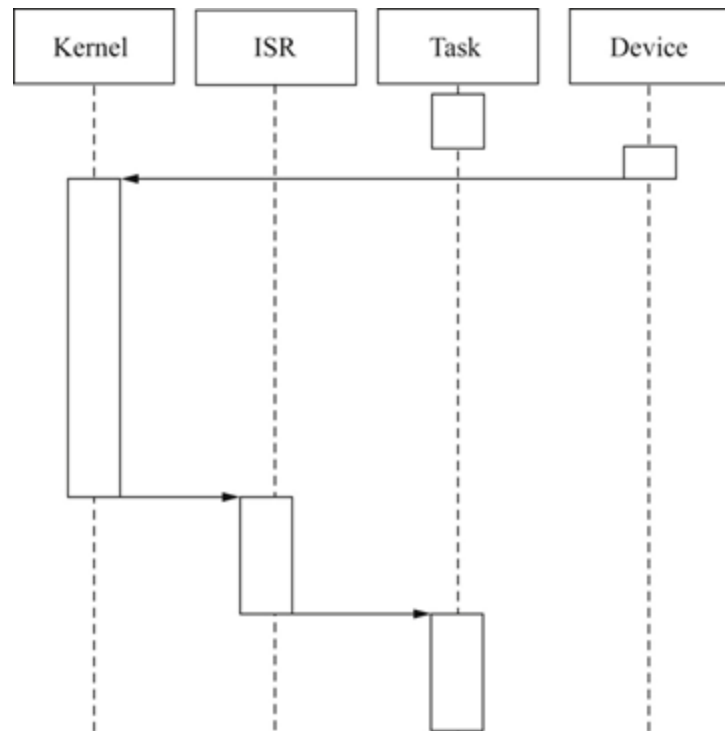
- We have assumed that we know the execution time of the processes.
- Execution time is not constant:
 - But can be bounded by **worst-case** and **best-case** execution times.
 - **We probably need to consider the worst-case!**

System Performance

- Context switching can **add significant delay** in some cases.
- Context switching time depends on several factors:
 - The amount of CPU context that must be saved
 - Scheduler execution time
 - *E.g., Earliest-Deadline-First scheduling, requires sorting the deadlines: $O(n \log n)$ activity*

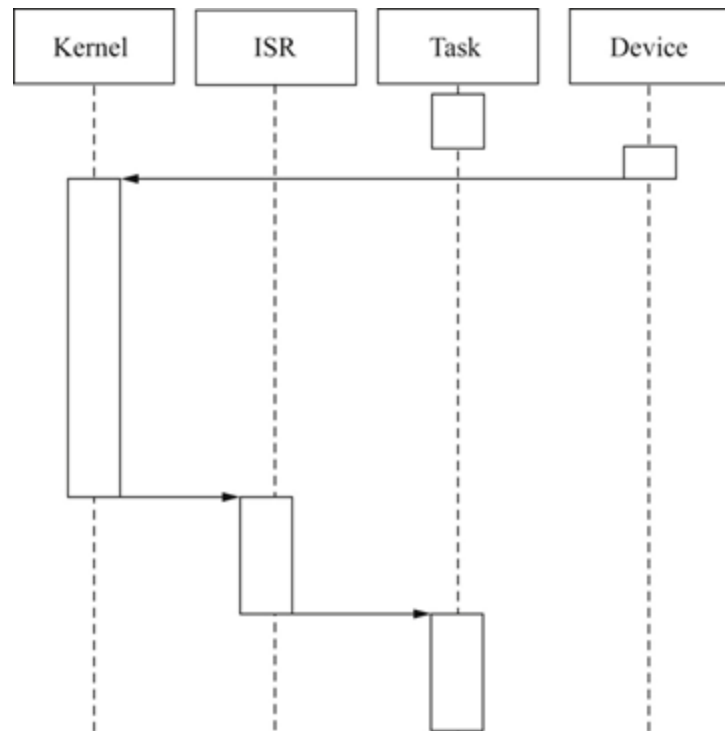
System Performance

- We have largely ignored **interrupt latency**:
 - The latency from when an interrupt is requested to when the device's service is complete is a critical parameter of real-time performance.
- Several factors affect:
 - The processor interrupt latency
 - The execution time of the interrupt handler
 - Delays due to RTOS scheduling.



System Performance

- A higher-priority interrupt may delay a lower-priority interrupt.
- Divide interrupt handling into two different pieces of code.
 - The interrupt service handler performs the minimal operations required to respond to the device.
 - A user-mode thread does the rest of the required processing



FreeRTOS Examples

Introduction to FreeRTOS

- [FreeRTOS.org](https://www.freertos.org): Real-time kernel
 - allows embedded applications to use **multitasking**.
- Tasks can be created with the `xTaskCreate()` function:

```
BaseType_t xTaskCreate(  
    TaskFunction_t pvTaskCode, // Pointer to the task function  
    const char * const pcName, // Descriptive name of the task  
    uint16_t usStackDepth,     // Stack size allocated for the task  
    void * pvParameters,       // Task specific parameters  
    UBaseType_t uxPriority,     // Priority of the task  
    TaskHandle_t * pxCreatedTask // Handle to the task  
);
```

```
void vTaskFunction ( void * pvParameters ){  
    // Initialization can be done here  
    ...  
    while (1) {  
        // Task functionality comes here  
        ...  
    }  
}
```

Introduction to FreeRTOS

- FreeRTOS.org: Real-time kernel
 - allows embedded applications to use **multitasking**.
- Tasks can be created with the `xTaskCreate()` function:

```
/* The handle to Task , which can be used to delete the task */
TaskHandle_t xTaskHandle = NULL ;

char * pcParameters1 = " Task is running and running .";

result = xTaskCreate (vTaskFunction , " MyTask " , 1000 , (void *) pcParameters1 , 1, &xTaskAHandle);

if (result != pdPASS) { "Error creating TaskA task" return 0;}
```

Introduction to FreeRTOS

- After creating the tasks, RTOS scheduler can be started by calling created with the **vTaskStartScheduler**

```
/* The handle to Task , which can be used to delete the task */  
TaskHandle_t xTaskHandle = NULL ;  
  
char * pcParameters1 = " Task is running and running .";  
  
result = xTaskCreate (vTaskFunction , " MyTask ", 1000 , (void *) pcParameters1 , 1, &xTaskAHandle);  
  
if (result != pdPASS) { "Error creating TaskA task" return 0;}  
  
vTaskStartScheduler();
```

Introduction to FreeRTOS

- Tasks can be deleted with the `vTaskDelete()` function:

```
/* The handle to Task , which can be used to delete the task */
TaskHandle_t xTaskHandle = NULL ;

char * pcParameters1 = " Task is running and running .";

result = xTaskCreate (vTaskFunction , " MyTask ", 1000 , (void *) pcParameters1 , 1, &xTaskAHandle);

if (result != pdPASS) { "Error creating TaskA task" return 0;}

vTaskStartScheduler();

...

vTaskDelete ( xTaskHandle );

...
```

Introduction to FreeRTOS

- Tasks can be **blocked** with the **vTaskDelay** function:

```
void vTaskFunction( void *pvParameters ){
    char *pcTaskName;
    const TickType_t xDelay250ms = pdMS_TO_TICKS(250);
    /* The string to print out is passed in via the parameter. Cast this to a character pointer. */
    pcTaskName = ( char * ) pvParameters;
    /* As per most tasks, this task is implemented in an infinite loop. */
    for( ;; ){
        /* Print out the name of this task. */
        uart_println( pcTaskName );
        /* Delay for a period. This time a call to vTaskDelay() is used which places the task into the
        Blocked state until the delay period has expired. The parameter takes a time specified in 'ticks',
        and the pdMS_TO_TICKS() macro is used (where the xDelay250ms constant is declared) to convert 250
        Milliseconds into an equivalent time in ticks. */
        vTaskDelay(xDelay250ms);
    }
}
```

Let's try our first example...

- Run FreeRTOS-HelloWorld.
 - Why the output does not appear properly, in order?

FreeRTOS Scheduling Algorithm

- FreeRTOS uses a scheduling algorithm called **Fixed Priority Preemptive Scheduling** with Time Slicing.
 - **Fixed Priority:** The scheduler does not change the priorities assigned to the scheduled tasks
 - *But tasks can change their priorities themselves*
 - **Preemptive:** The higher-priority task always preempts the lower priority.
 - **Time Slicing:** Shares processing time between tasks of equal priority

Shared Resources and Race Condition

- A task may need to do more than read and write values to and from memory.
- E.g., it may need to communicate with an I/O device
 - I/O devices are shared among tasks
- A task may use shared memory locations to communicate with other tasks.
 - E.g., through shared variables

Shared Resources and Race Condition

- When dealing with **shared resources**, special care must be taken:
 - Problem: *What happens when two tasks try to write the same location?*
- E.g., A flag variable is used to manipulate a variable.
 - Task 1 reads the flag and sees 0.
 - Task 2 reads the flag and sees 0.
 - Task 1 sets the flag to one and writes the location.
 - Task 2 sets the flag to one and overwrites the location.

Critical Sections

- **Critical section:**

- Section of code that **cannot be interrupted** by another task.

- **Examples:**

- Writing shared memory locations

- Accessing I/O device.

- *We need to be sure that a task finishes an I/O operation before allowing another task to start its own operation on that I/O device.*

Kernel Objects

- To prevent race conditions and implement critical sections, we need **kernel objects**:
 - Facilitate **synchronization** and **communication** between two or more tasks (threads) of execution.
- Examples of such objects include **semaphores, message queues, signals, pipes,...**

Semaphores

- A kernel object that one or more tasks can **acquire** or **release**:
 - E.g., to guard a resource.
- Protocol:
 - Get access to semaphore: **wait()**
 - Perform critical region operations.
 - Release semaphore: **post()**

```
/* some nonprotected operations here */  
W(); /* wait for semaphore */  
... /* do protected work here */  
P(); /* release semaphore */
```

Semaphores

- To implement wait() and post(), the microprocessor bus must support an **atomic read/write** operation
 - A single bus operation reads a memory location, tests it, and writes it.
- ARM **test-and-set** provided by **SWP**:

```
                ADR r0, SEMAPHORE      ; get semaphore address
                LDR r1, #1
GETFLAG SWP r1,r1,[r0]                ; test-and-set the flag
                BNZ GETFLAG             ; no flag yet, try again
HASFLAG ...
```

Mutual Exclusion (Mutex) Semaphores

- A special **binary semaphore**:
 - the states of a mutex are unlocked or locked (0 or 1, respectively).
 - Mutex **has an owner**: only the owner can unlock
 - *A binary semaphore does not have an owner, any task can release the semaphore*
 - No ownership check!

FreeRTOS Semaphore/Mutex

- The creation of a **binary** or **counting semaphore**:
 - xSemaphoreCreateBinary or xSemaphoreCreateCounting
- The creation of a **mutex**
 - xSemaphoreCreateMutex

```
// Function to create a new binary semaphore and get its handle  
SemaphoreHandle_t xSemaphoreCreateBinary (void);
```

```
// Function to create a new counting semaphore and get its handle  
SemaphoreHandle_t xSemaphoreCreateCounting (UBaseType_t  
uxMaxCount, UBaseType_t uxInitialCount);
```

```
// Function to create a new mutex and get its handle  
SemaphoreHandle_t xSemaphoreCreateMutex (void);
```

FreeRTOS Semaphore/Mutex

- To take and release semaphores:
 - `xSemaphoreTake` and `xSemaphoreGive`
 - `xSemaphoreGiveISR` to release inside an **interrupt service routine**.

```
// Function to take the semaphore (mutex)
```

```
xSemaphoreTake (SemaphoreHandle_t xSemaphore, // Handle to the semaphore (mutex)  
                TickType_t xTicksToWait );    // Number of ticks to wait before function returns  
automatically
```

```
// Function to return the semaphore (mutex)
```

```
xSemaphoreGive (SemaphoreHandle_t xSemaphore); // Handle to the semaphore (mutex)
```

```
// Function to return the semaphore (mutex) inside ISR
```

```
xSemaphoreGiveFromISR(SemaphoreHandle_t xSemaphore, // Handle to the semaphore (mutex)  
                      BaseType_t *pxHigherPriorityTaskWoken); // TRUE if the unblocked task has a priority
```

```
// higher than the currently executing task
```

FreeRTOS Semaphore/Mutex

- LAB - Mutex Example: Controlled access to UART (serial port)
 - Check if the resource is accessed properly!

```
void vTaskAFunction(void *pvParameters)
{
    char *message = (char*)pvParameters;
    uart_println("Start TaskA.");

    while (1) {
        xSemaphoreTake( xMutex, portMAX_DELAY );
        uart_println(message);
        xSemaphoreGive( xMutex );
        vSimpleDelay();
    }
}
```

```
void vTaskBFunction(void *pvParameters)
{
    char *message = (char*)pvParameters;
    uart_println("Start TaskB.");

    while (1) {
        xSemaphoreTake( xMutex, portMAX_DELAY );
        uart_println("%d: ",counter);
        uart_println(message);
        xSemaphoreGive( xMutex );

        ...
    }
}
```

FreeRTOS Semaphore/Mutex

- LAB - Counting Semaphore Example
 - Activity Synchronization (Event triggering)

```
xSemaphoreA = xSemaphoreCreateCounting(1,1);  
xSemaphoreB = xSemaphoreCreateCounting(1,0);
```

```
void vTaskAFunction(void *pvParameters)  
{  
    char *message = (char*)pvParameters;  
    uart_println("Start TaskA.");  
  
    while (1) {  
        xSemaphoreTake( xSemaphoreA, portMAX_DELAY );  
        uart_println(message);  
        xSemaphoreGive(xSemaphoreB);  
        vSimpleDelay();  
    }  
}
```

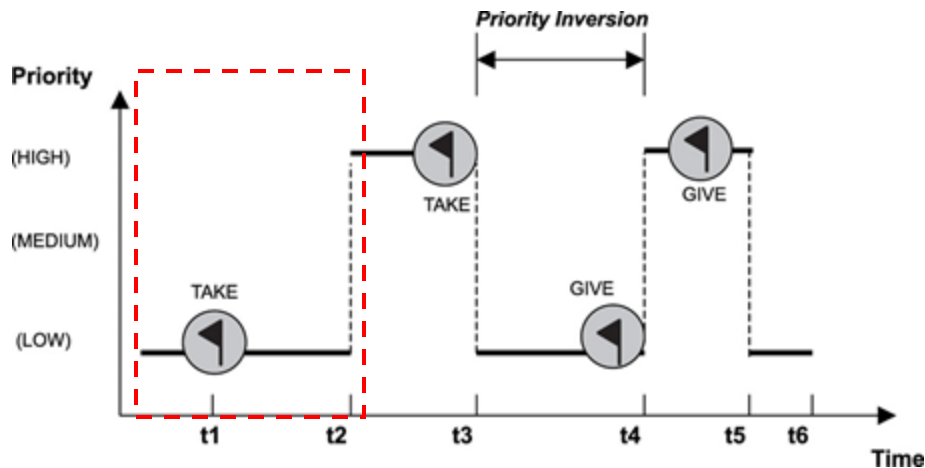
```
void vTaskBFunction(void *pvParameters)  
{  
    char *message = (char*)pvParameters;  
    uart_println("Start TaskB.");  
  
    while (1) {  
        xSemaphoreTake(xSemaphoreB, portMAX_DELAY );  
        uart_println("%d: ",counter);  
        uart_println(message);  
        xSemaphoreGive(xSemaphoreA);  
        ...  
    }  
}
```

Priority Inversion

- A low-priority task **executes** while a higher-priority task **waits** on it due to resource contentions.
- Improper use of system resources can cause scheduling problems:
 - Low-priority process grabs I/O device.
 - High-priority device needs the I/O device but can't get it until low-priority process is done.
 - Can cause **deadlock**.

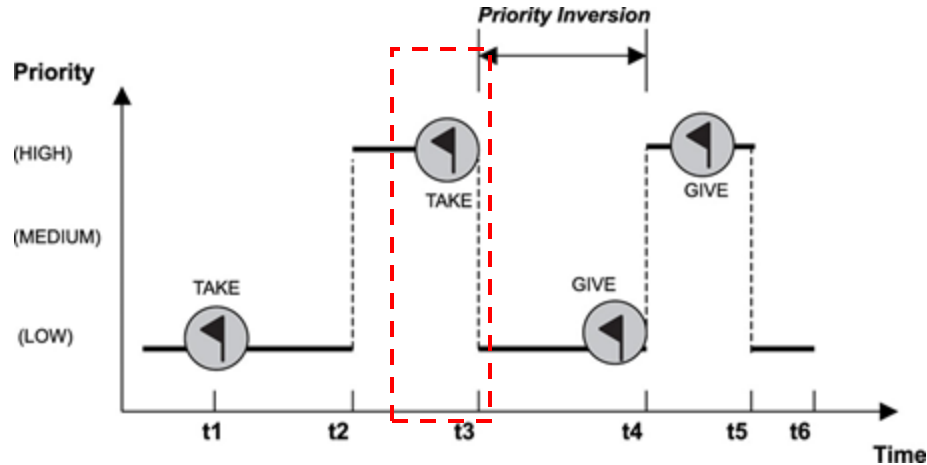
Priority Inversion

- t1: Low-priority task **locks** the shared resource.
- t2: High-priority task becomes eligible to run.
 - Context switch to the high-priority task.



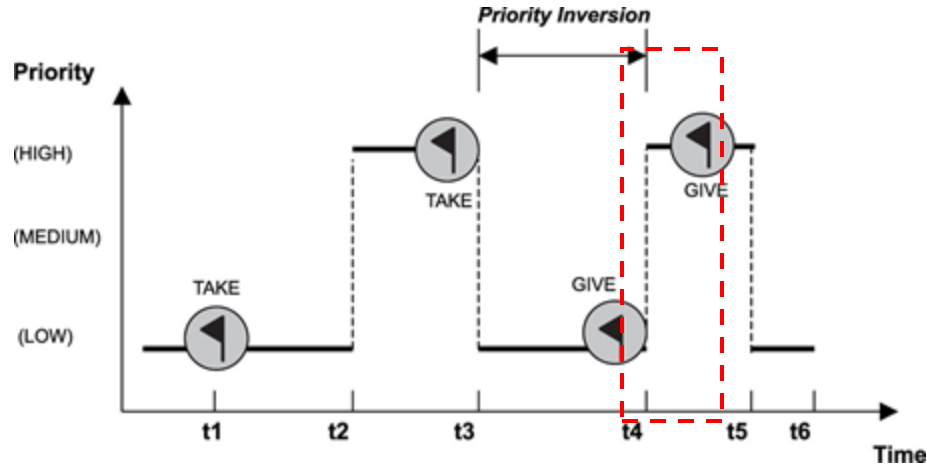
Priority Inversion

- t3: High-priority task requires the shared resource
 - Context-switch back to the low-priority task.
 - **Priority inversion** begins at time t3



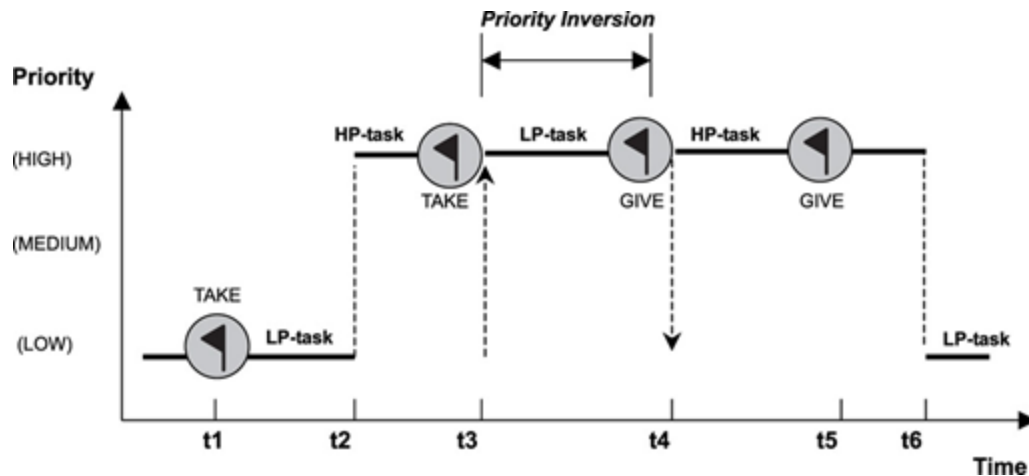
Priority Inversion

- t4: Low-priority task releases the shared resource
 - Context-switch to high-priority task.
 - **Priority inversion** ends at time t4.



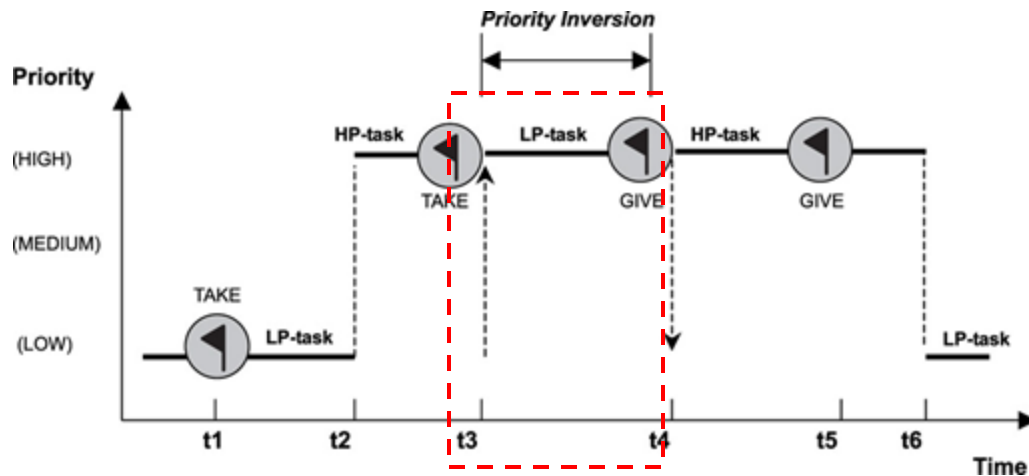
Priority Inheritance

- A common method for dealing with priority inversion:
 - Raises the priority of a low-priority task **temporarily** to the same priority level as the higher-priority task
 - Once the process is finished with the resource, its priority is **demoted** to its normal value



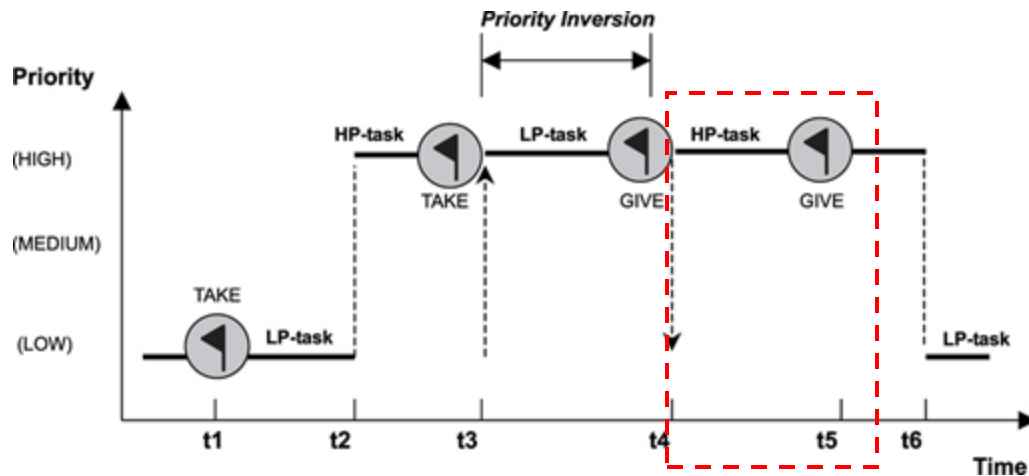
Priority Inheritance

- t3: when the low-priority task blocks the high-priority task:
 - The execution priority is **raised** to that of the high-priority task.
 - *Ensures that medium-priority tasks cannot interfere*



Priority Inheritance

- t4: when the low-priority task releases control of the shared resource:
 - The priority is immediately **lowered** to its previous level:
 - *allows the high-priority task to preempt its execution.*



Message Queues

- A message queue is a **buffer-like** object:
 - Tasks and ISRs send and receive messages to communicate and synchronize
- This temporary buffering **decouples** a sending and receiving task

FreeRTOS: Message Passing Via Queues

- The queue uses a FIFO policy and:
 - described by the data type `xQueueHandle`
 - created using `xQueueCreate`.
- A message is put into the queue using `xQueueSend()` and received using `xQueueReceive()`

```
xQueueHandle q1;  
q1 = xQueueCreate(MAX_SIZE,sizeof(msg_record)); /* maximum number of records in queue, size of each record */  
if (q1 == 0) /* error */  
...  
xQueueSend(q1,(void *)msg,(portTickType)0); /* queue, message to send, final parameter controls timeout */  
if (xQueueReceive(q2,&(in_msg),0); /* queue, message received,timeout */  
...
```

Critical Sections

- Interrupts are masked during the **critical section**:
 - The timer cannot interrupt and other processes cannot start to execute.
 - The kernel may also have its own critical sections
- Keep interrupts from being serviced and other processes from executing.
 - Long critical sections cause major problems for interrupts.